TU Clausthal

Master's Thesis

by

Yu Zhang

Explain Reality: Grounded Procedural Tutoring with Foundation Models and Visual Fact Extraction in Immersive Simulation

1st Supervisor:	**Prof. Dr. Christian Bartelt**
2nd Supervisor:	**Prof. Dr. Rüdiger Ehlers**

9th June 2026

Institute for Software and Systems Engineering
Clausthal University of Technology

Declaration of Authorship

I have read and understood the guidelines of the Technical University of Clausthal and those published in the ISSE bulletin, including those regarding the use of literature and other sources. I confirm that I have prepared this thesis independently by myself. Any information taken from other sources and being reproduced in this thesis is clearly referenced.

In terms of the general examination regulations, this work has not yet been submitted to any other examination division.

I hereby agree that my master's thesis may be exhibited in the institute's library and kept for inspection.

Clausthal-Zellerfeld, 9th June 2026	
Location, Date	Yu Zhang

Abstract

Many safety- and skill-critical domains, including aviation, industrial maintenance, medicine, and emergency operations, require learners to practise ordered procedures whose next correct action depends on the current system state. As such training moves into high-fidelity and immersive simulation, tutoring systems must provide contextual help without relying on language-model fluency alone. This thesis studies that problem through the F/A-18C cold-start procedure in DCS World and designs an evidence-grounded tutoring runtime that integrates procedure-pack-driven state tracking, retrieval-augmented help generation, and a fine-tuned vision-language model (VLM) for structured visual fact extraction from cockpit displays. A central finding is that visual fact ontology design governs model reliability: evolving from an 8-fact to a 13-fact ontology by decomposing compound targets into locally verifiable visual atoms reduced critical false positives by 64–89% on independent holdout sets. A Qwen3.5-9B LoRA adapter trained on 928 bilingual rows achieves 0.99 fact accuracy on two holdout sets with verified zero training overlap; the identical data recipe transfers to Gemma-4-31B with consistent gains, suggesting the approach is not tied to a single backbone. A small formative VR pilot study with nine participants provides descriptive evidence: all four tutor-condition participants completed the full 33-step procedure, whereas none of the five without-tutor participants did; omission errors were largely replaced by assistance-coupled errors in the tutor condition. Controlled evaluation with larger samples remains future work.

Zusammenfassung

Prozedurales Lernen in hochrealistischer Simulation erfordert das Ausführen geordneter, zustandsabhängiger Handlungsabfolgen bei gleichzeitiger Überprüfung von Systemzuständen. Herkömmliche Trainingshilfen unterbrechen jedoch entweder die Immersion oder haben keinen Zugriff auf den Simulatorzustand. Diese Arbeit begegnet dieser Lücke durch eine telemetriegestützte Tutoring-Laufzeitumgebung, die paketbasierte Zustandsverfolgung, Retrieval-Augmented-LLM-Hilfegenerierung und ein feinabgestimmtes Vision-Language-Model (VLM) zur strukturierten Extraktion visueller Fakten aus Cockpit-Anzeigen kombiniert. Ein zentrales Ergebnis ist, dass das Design der visuellen Fakten-Ontologie die Modellzuverlässigkeit bestimmt: Die Weiterentwicklung von einer 8-Fact- zu einer 13-Fact-Ontologie durch Zerlegung zusammengesetzter Ziele in lokal überprüfbare visuelle Atome reduzierte kritische False-Positives um 64–89% auf unabhängigen Holdout-Sets. Ein Qwen3.5-9B LoRA-Adapter, trainiert mit 928 bilingualen Datenzeilen, erreicht eine Fact-Genauigkeit von 0,99 auf zwei Holdout-Sets mit verifizierter Null-Überlappung zum Training; dieselbe Datenrezeptur überträgt sich mit konsistenten Verbesserungen auf Gemma-4-31B, was darauf hindeutet, dass der Ansatz nicht an ein einzelnes Backbone gebunden ist. Eine kleine formative VR-Pilotstudie mit neun Teilnehmern liefert deskriptive Evidenz: Alle vier Teilnehmer mit Tutor absolvierten die vollständige 33-Schritt-Prozedur, während keiner der fünf Teilnehmer ohne Tutor dies schaffte; Auslassungsfehler wurden in der Tutor-Bedingung weitgehend durch assistenzgekoppelte Fehler ersetzt. Eine kontrollierte Evaluation mit größeren Stichproben bleibt zukünftige Arbeit.

Acknowledgements

I would like to thank my supervisor for guidance and feedback throughout this thesis. I also thank the pilot study participants for their time, patience, and careful engagement during the VR sessions.

Declaration on the Use of AI Tools

AI-assisted tools, including ChatGPT/Codex and Claude Code, were used during the preparation of this thesis for language editing, restructuring suggestions, LaTeX consistency checks, citation-gap auditing, and inspection of repository artefacts under the author's direction. These tools were not used to fabricate experimental data, participant results, or bibliographic metadata. All claims, numerical results, citations, interpretations, and final wording remain the responsibility of the author.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Motivation and Problem Context

Many safety- and skill-critical domains require people to learn ordered procedures whose next correct action depends on the current state of a system. Examples include aircraft cockpit procedures, industrial maintenance, emergency operations, and medical or technical equipment workflows. In such settings, procedural knowledge is not only a list of actions. It also includes preconditions, intermediate checks, state-dependent branching, and recovery from partially completed or incorrectly completed steps.

High-fidelity and immersive simulation make these procedures available for practice without exposing learners to the risks and costs of the real environment. At the same time, fidelity creates a tutoring problem. A learner who is already inside a simulated cockpit, machine, or procedure room may need help that is specific to the current state rather than a general explanation of the task. If the learner has to leave the simulation to consult a manual, video, or checklist, the training flow is interrupted. If the learner instead asks a language model for help, the answer may be fluent but disconnected from the state that actually exists in the simulation.

This thesis studies that problem through the F/A-18C cold-start procedure in DCS World. The flight-simulation setting is used as a concrete and demanding case study, not as a claim that the research problem exists only in aviation. The case is useful because it makes the evidence problem visible: procedure progress depends on switch states, engine parameters, warning lights, cockpit display pages, and the order in which earlier actions were completed. Some of these states are directly observable through telemetry; others require visual inspection of cockpit displays; and a small number require manual confirmation or lie outside the display layout used by the vision pipeline.

The current procedure pack contains 33 ordered steps (S01–S33) grouped into six operational phases, from initial electrical power through engine start, avionics configuration, flight-control checks, and final pre-taxi configuration (Section 2.1). The benchmark and VLM evaluation reported in this thesis focus on the original 25-step core procedure;

the later extension to S33 adds further pre-taxi checks without changing the central evidence problem. This makes the cold-start a compact but representative case for state-dependent procedural tutoring in high-fidelity simulation.

1.2 Research Gap and Problem Statement

Existing training aids and recent foundation-model systems each address part of the problem but leave an important gap. Manuals, videos, and checklists provide authoritative procedural knowledge, yet they do not observe the learner's current simulator state. Conventional simulator telemetry can expose many states, yet it does not by itself explain what the learner should do next or interpret display content that is not exported as data. Foundation models can generate flexible explanations, but their output is not reliable enough for procedural guidance unless it is constrained by explicit evidence.

The central premise of this thesis is that procedural assistance becomes more reliable and more auditable when every actionable tutoring output is tied to an explicit evidence source. In the system developed here, such evidence may be a telemetry variable, a procedure-gate evaluation, a retrieved manual or checklist snippet, or a VLM-derived visual fact. The language model is therefore not asked to infer procedure state from parametric knowledge alone. It operates inside a runtime that supplies state evidence, checks structured outputs, validates overlay targets, and records the resulting help cycle for replay and analysis.

This framing changes the role of the foundation model. The model is not the tutor by itself; it is one component inside an evidence-grounded procedural tutoring runtime. The research question is therefore architectural and methodological as much as it is generative: how should simulator telemetry, procedure rules, retrieval, and visual fact extraction be connected so that generated help remains state-aware, inspectable, and bounded by available evidence?

1.3 Research Questions

The thesis is guided by one main research question and three subordinate questions:

Main RQ. How can an evidence-grounded multimodal tutoring system support state-dependent procedural training in high-fidelity immersive simulation?

RQ1 — System architecture and evidence integration. How can simulator telemetry, retrieved procedural knowledge, and visual fact observations be integrated so that tutoring outputs are state-aware, auditable, and safe for procedural guidance?

RQ2 — VLM adaptation and ontology design. To what extent can a small-data LoRA-adapted VLM reliably extract structured cockpit visual facts, and how does visual fact ontology design affect critical false positives?

RQ3 — Formative VR pilot evidence. In a VR-based F/A-18C cold-start training task, what formative evidence does a small pilot study provide about task completion, procedural errors, and tutor interaction in a with-tutor versus without-tutor comparison?

RQ1 and RQ2 are addressed through the implemented runtime, replay and harness infrastructure, and technical benchmark evaluation. RQ3 is addressed through a completed formative VR pilot study ($n = 9$). The pilot is treated as descriptive evidence about feasibility, instrumentation, completion, and error patterns; it is not used to claim controlled learning gains, workload reduction, retention, or transfer.

1.4 Approach and Scope

The thesis addresses the research questions by building and evaluating an evidence-grounded tutoring runtime for the F/A-18C cold-start case. The approach combines four evidence channels. First, DCS-BIOS telemetry provides the most authoritative signal for many cockpit controls, lights, and numerical engine or system values. Raw UDP frames are normalised, sanitised, and aggregated into stable runtime variables so that procedure gates can distinguish between a false value and a missing source. Second, where telemetry is insufficient, a visual fact layer converts cockpit display captures into structured facts with states `seen`, `not_seen`, or `uncertain`. Third, retrieved procedural sources provide textual support for explanations. Fourth, procedure-pack gates define which evidence is required before a step can be considered available or complete.

The help cycle is deliberately constrained. The runtime assembles an evidence packet containing the current procedure candidates, telemetry state, visual observations, and retrieved sources before model generation occurs. The LLM/VLM output is then validated against runtime-generated JSON Schema constraints and mapped through allowlisted overlay targets. If the model path is unavailable or invalid, the system falls back to deterministic, text-only guidance. This design makes the help cycle auditable: a later

replay can inspect which evidence was available, which step was inferred, what the model returned, and how the response was validated before display.

The scope is intentionally bounded. The thesis validates the architecture and visual fact pipeline technically, and it reports a completed formative VR pilot study. It does not claim that the system has established controlled learning gains, workload reduction, retention, transfer, or generality across all procedural domains. The F/A-18C case is used to make the evidence-grounding problem concrete and measurable. Extending the approach to other aircraft, industrial equipment, medical training tasks, or emergency procedures would require new procedure packs, evidence mappings, and domain-specific validation.

Within that boundary, the thesis scope is threefold:

1. The design and implementation of an evidence-grounded tutoring runtime that combines DCS-BIOS telemetry, procedure-pack gates, retrieved procedural knowledge, VLM-derived visual facts, structured response validation, overlay display, logging, and replay (Chapters 3, 4, and 5);

2. The technical validation of the visual fact adaptation pipeline through independent holdout benchmarks and cross-backbone comparisons (Chapter 6, Part A);

3. A completed formative VR pilot study ($n = 9$) that provides descriptive evidence about task completion, error patterns, tutor interaction, and instrumentation readiness under with-tutor and without-tutor conditions (Chapter 6, Part B).

Procedure definitions, gates, telemetry mappings, visual fact bindings, and error taxonomies are kept in declarative procedure-pack configuration rather than embedded in prompts. This pack-based design keeps the architecture aligned with Clean / Hexagonal principles: domain rules remain separate from simulator adapters, vision adapters, model adapters, and export tooling.

1.5 Contributions

This thesis makes three contributions.

1. **An evidence-grounded procedural tutoring architecture.** The thesis presents a ports-and-adapters tutoring runtime in which each actionable help output must be traceable to telemetry, a procedure-gate evaluation, a retrieved source, or a

visual fact. The contribution is not merely that an LLM is connected to a simulator. It is that generated tutoring is placed inside a constrained help cycle with declarative procedure packs, schema-validated responses, allowlisted overlay targets, deterministic fallbacks, and replayable logs. This contribution addresses RQ1 and is described in Chapters 3 and 5, with validation evidence in Chapter 6, Section 6.2.

2. **A visual fact ontology and small-data VLM adaptation pipeline.** The thesis develops a 13-fact visual ontology for F/A-18C cockpit display state, evolved from an 8-fact baseline that exposed the weakness of asking a VLM to recognise compound procedural states directly. The revised ontology decomposes such targets into locally observable visual atoms and leaves procedural interpretation to downstream rules. The associated data pipeline spans DCS screenshot capture, AI pre-labelling, human review, bilingual SFT row generation, LoRA fine-tuning, and independent holdout benchmarking. On the Qwen backbone, a LoRA adapter trained on 928 bilingual rows reaches approximately 0.99 fact accuracy on two independent holdouts and reduces critical false positives by 64–89%; the same data recipe also improves the Gemma backbone. This contribution addresses RQ2 and is described in Chapters 3 and 4, with evaluation in Chapter 6, Sections 6.1.2 and 6.1.3.

3. **A VR procedural training evaluation framework and formative pilot study.** The thesis delivers the instrumentation required to study evidence-grounded procedural tutoring with real participants: experiment export tooling, baseline and tutor-condition configurations, session launch support, replayable logs, semi-automated pre-scoring, error coding guides, and an experiment operation manual. A formative VR pilot study ($n = 9$) was conducted with four with-tutor and five without-tutor participants. All four tutor-condition participants completed the full 33-step procedure; none of the five without-tutor participants did. The observed error patterns also differed: tutor-condition errors were mainly assistance-coupled, while without-tutor errors were mainly omissions. These results support feasibility and instrumentation claims, but not controlled claims about learning effectiveness. This contribution addresses RQ3 and is documented in Chapter 6, Section 6.3, and Appendix 9.

Taken together, these contributions define the boundary of the thesis. The work demonstrates that evidence-grounded tutoring can be engineered, benchmarked, and

exercised in a formative VR study for a demanding simulator procedure. It does not claim that educational effectiveness has been established beyond that formative evidence.

1.6 Thesis Structure

Chapter 2 explains why the F/A-18C cold-start task requires multiple evidence sources, introduces the visual fact ontology, and positions the thesis against related work in intelligent tutoring, simulator instrumentation, state-aware assistance, VLM adaptation, and grounded AR/VR systems.

Chapter 3 presents the evidence-grounded architecture, including the ports-and-adapters structure, runtime data flow, procedure and gating model, telemetry processing, replay infrastructure, and VLM component.

Chapter 4 describes the methods used to create and validate the evidence chain: telemetry grounding, knowledge retrieval and source policy, visual fact data collection, ontology evolution, LoRA fine-tuning, and holdout benchmark design.

Chapter 5 documents the implementation choices that make the architecture operational, including core domain logic, adapter boundaries, structured help generation, overlay mapping, VLM integration, and experiment tooling.

Chapter 6 evaluates the thesis claims in two parts: technical evidence for RQ1 and RQ2, followed by the formative VR pilot evidence for RQ3.

Chapter 7 interprets the results, draws design lessons from the ontology revision, discusses scope boundaries, and states the limitations and future work.

Chapter 8 summarises the completed work against the research questions and restates the claim boundaries of the thesis.

Appendix 9 contains supplementary figures, benchmark tables, dataset statistics, hyperparameter configurations, CLI references, and the 8-fact baseline benchmark.

2 Background and Related Work

The previous chapter framed the thesis around one requirement: generated procedural help should be grounded in evidence that can be inspected before and after it reaches the learner. This chapter explains why that requirement arises in the F/A-18C cold-start task and how it relates to existing work. The first part introduces the task, the available evidence channels, and the visual fact ontology used to turn cockpit screenshots into structured observations. The second part positions the thesis against intelligent tutoring systems, simulator-based training, rule-based state tracking, VLM adaptation, and grounded assistance in AR/VR.

2.1 The F/A-18C Cold-Start Task

The procedural scenario studied in this thesis is the F/A-18C cold-start in DCS World, a pre-flight cockpit procedure that moves the aircraft from a cold and dark state to a taxi-ready configuration [7, 8]. The task is useful as a research case because it is both bounded and state-sensitive. It has a known order of actions, yet successful completion depends on intermediate states that cannot be reduced to a text checklist: electrical power must be established before engine start, engine parameters must enter nominal ranges before the second engine is engaged, cockpit displays must show the expected pages, and some checks only make sense after earlier system transitions have occurred.

For a tutoring system, the relevant property of the cold-start is therefore not only its length. It is the way the procedure forces the tutor to combine three kinds of evidence: sequence evidence about which steps have been completed, telemetry evidence about cockpit controls and instruments, and visual evidence about display-page content. A response that ignores any of these signals can be locally fluent but procedurally wrong.

2.1.1 Procedure Structure: S01–S33 and Phases P1–P6

The current procedure pack represents the cold-start as 33 numbered steps (S01–S33) grouped into six phases. The phase structure is used in the thesis as a compact descrip-

tion of the evidence demands imposed by the task:

P1 (S01–S03): Power-up and safety checks. Battery and generator activation, fire-detection tests, and APU start establish the first electrical and safety preconditions for the rest of the procedure.

P2 (S04–S07): Right engine start. Right engine crank, throttle advance, bleed-air cycling, and caution/warning/advisory light testing introduce telemetry-dependent timing and verification.

P3 (S08–S09): Displays and communications. The DDIs, AMPCD, HUD, display pages, and COMM1 preset configuration make display state part of the procedural evidence chain.

P4 (S10–S14): Left engine and core systems. Left engine start, INS alignment mode selection, radar power, and OBOGS activation depend on earlier engine and avionics state.

P5 (S15–S27): FCS and flight controls. FCS reset and BIT, flap/trim configuration, launch-bar and arrestor-hook checks, refuelling probe checks, and pitot heat configuration mix telemetry-visible controls with display-based confirmation.

P6 (S28–S33): Pre-taxi instruments and final checks. Parking brake, BINGO fuel, altimeter settings, standby attitude indicator uncaging, and attitude-source selection complete the taxi-ready state.

Each step is represented in the procedure pack with precondition and completion gates. These gates make explicit which previous steps and which state variables must be satisfied before the tutor can treat a step as available or complete. For example, the right engine crank step depends on APU support, while the left engine start depends on right-engine parameters such as RPM, temperature, fuel flow, nozzle position, and oil pressure entering nominal ranges. This is why the task is a useful test case for evidence-grounded help: a tutor that advances the learner based on checklist order alone can miss the actual aircraft state.

2.1.2 Cockpit Displays and the Composite Panel

Many cold-start states are visible on the F/A-18C's cockpit displays rather than directly available as simple telemetry variables. The visual pipeline therefore operates on a fixed composite panel image containing three display regions:

- **Left DDI (Digital Display Indicator):** used during the procedure for pages such as FCS, SUPT, and TAC. In the evaluated cold-start flow, the FCS page and FCS page markings are important visual confirmations.
- **AMPCD (Advanced Multipurpose Color Display):** used for the HSI page, map layer, and INS alignment text. These states are central to the alignment-related visual facts.
- **Right DDI:** used for the BIT root page and FCS-MC BIT sequence, including intermediate, in-test, and final GO states.

Using a composite image keeps training and inference aligned: the model sees the same three-display layout during dataset construction, benchmarking, and runtime capture. More importantly, the composite panel defines a narrow perception boundary. The VLM is not asked to interpret the whole VR scene or infer the pilot's intention. It only reports whether specific cockpit-display facts are visible in known regions.

2.1.3 DCS-BIOS Telemetry

DCS-BIOS provides the main non-visual evidence source for the runtime. It exports cockpit switch positions, indicators, warning lights, gauges, and other instrument states from DCS World through a UDP telemetry stream [6]. The tutoring runtime normalises this stream into stable variables such as `rpm_r`, `apu_ready`, and `ins_mode`, applies delta sanitisation to reduce jitter, and tracks whether a source is missing rather than merely false.

This telemetry channel is authoritative where it exists. Steps such as battery power, throttle movement, radar mode, and many final pre-taxi switch states can be checked without computer vision. At the same time, DCS-BIOS does not export all display-page content needed by the cold-start tutor. The runtime therefore uses vision only for states that telemetry cannot observe, such as whether the FCS page, BIT root page, FCS-MC BIT result, or INS alignment text is visible. Figure 2.1 summarises this division of evidence across the 33-step procedure.

2.2 Visual Fact Ontology

The visual fact ontology is the contract between perception and procedural reasoning. Rather than asking a VLM to produce a tutor decision such as "the learner should press

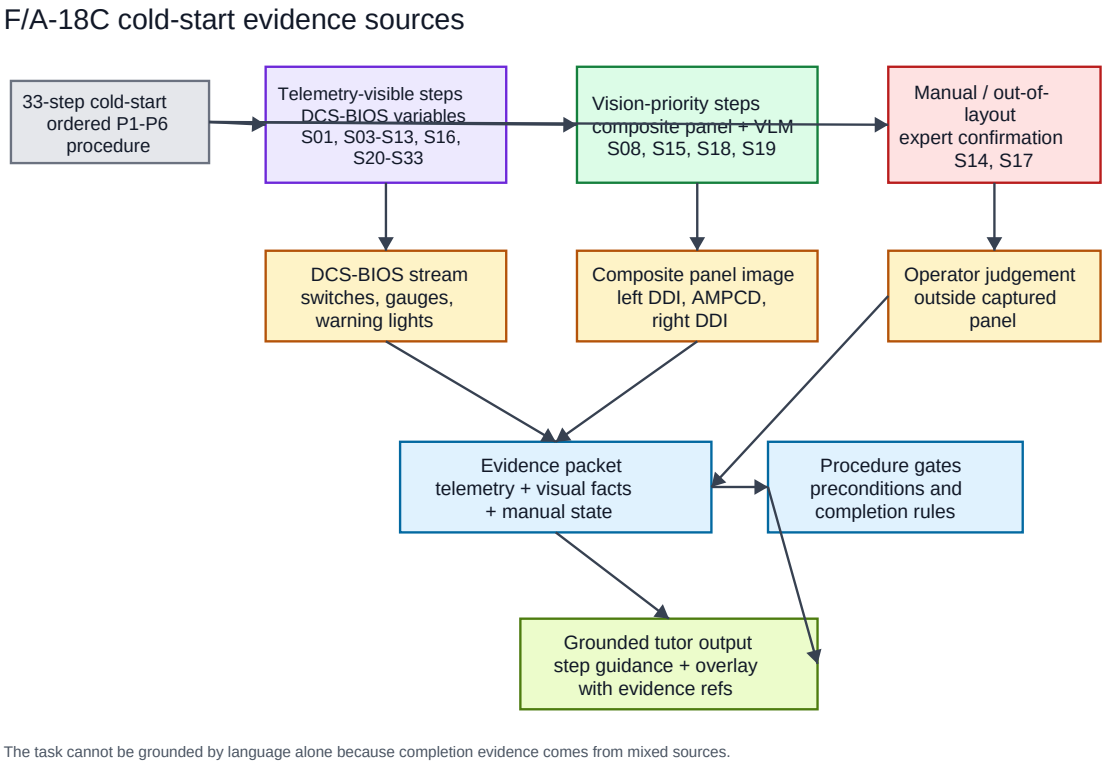

Figure 2.1: Evidence sources for the F/A-18C cold-start task. The 33-step procedure combines telemetry-visible steps, vision-priority display checks, and a small number of manual or out-of-layout confirmations; downstream tutoring decisions are grounded in the corresponding evidence source rather than in language-model output alone.

this button next", the runtime asks it to produce a small set of structured propositions about the visible cockpit display state. Each fact is a discrete observation with one of three values:

`seen`: the specified visual evidence is clearly present;

`not_seen`: the specified visual evidence is absent or not discernible;

`uncertain`: the image quality, occlusion, crop, or display state is insufficient for a confident decision.

This representation intentionally avoids mixing perception with task interpretation. The VLM reports whether visual evidence is visible. The procedure engine later combines those facts with telemetry, sequence state, and step bindings to decide whether a step is confirmed, pending, or blocked. The ontology is therefore not a minor implementation detail; it defines what kind of claim the model is allowed to make.

2.2.1 From 8-Fact v1 to 13-Fact v2

The ontology evolved through two generations. The first version contained eight facts over the left DDI, right DDI, and AMPCD. It included useful display-level targets such as FCS page visibility, BIT page visibility, FCS-MC page state, and INS alignment page visibility. It also included compound targets such as `ins_go`, which asked the model to infer a high-level procedural completion state from a single visual frame.

That design produced a clear lesson. The first LoRA adapter, trained on 180 human-reviewed Run-001 images, improved in-distribution behaviour but produced critical false positives on the independent Run-002 holdout: 13 of 50 samples were predicted as `seen` for `ins_go` when the human label was `not_seen`. The error was not only a model failure. It exposed a design problem in the target itself: `ins_go` combined visual perception with procedural interpretation.

The second-generation ontology replaces such compound targets with 13 observable visual atoms:

Region	Visual fact
Left DDI	`tac_page_visible`
Left DDI	`supt_page_visible`
Left DDI	`fcs_page_visible`
Left DDI	`fcs_page_x_marks_visible`
Right DDI	`bit_root_page_visible`
Right DDI	`fcsmc_page_visible`
Right DDI	`fcsmc_intermediate_result_visible`
Right DDI	`fcsmc_in_test_visible`
Right DDI	`fcsmc_final_go_result_visible`
AMPCD	`hsi_page_visible`
AMPCD	`hsi_map_layer_visible`
AMPCD	`ins_grnd_alignment_text_visible`
AMPCD	`ins_ok_text_visible`

The important change is not just the larger number of labels. The ontology moves from procedure-level judgement to observable evidence. For example, the v1 `ins_go` target is decomposed into AMPCD text and map-layer observations shown in the table above. The procedure engine then decides how these observations relate to the alignment step. This design principle carries the main scientific insight of the VLM part of the thesis: VLMs should extract local visual evidence, while procedural meaning should be assigned by explicit downstream rules.

2.2.2 Sticky Facts and Step Bindings

Two runtime mechanisms make the ontology usable during a live help cycle.

Sticky facts. Some visual events are transient but procedurally important. For example, the `fcsmc_final_go_result_visible` fact is marked as sticky with a time-to-live of 600 seconds. Once it has been observed as `seen`, later frames marked `not_seen` or `uncertain` do not immediately erase the evidence. This prevents a completed visual event from disappearing merely because the learner changed pages, the display flickered, or the capture window missed the relevant moment. Non-sticky facts, such as page visibility facts, expire quickly and are re-evaluated continuously.

Step bindings. Visual facts become procedurally meaningful only through explicit bindings in `vision_facts.yaml`. A binding specifies which facts must be visible for a step to be visually confirmed. Some bindings require all listed facts, such as S08 requiring both the FCS page on the left DDI and the BIT root page on the right DDI.

Others allow one of several alternatives when multiple visual cues can support the same procedural state.

The procedure pack also classifies each step by its preferred evidence source:

- `bios_priority_steps` for telemetry-visible states;
- `vision_priority_steps` for display-dependent states;
- `manual_or_out_of_layout_steps` for states requiring manual confirmation or lying outside the three-display composite.

This classification is the practical expression of the thesis's evidence-grounding principle: each step is checked against the most reliable available signal before it is used in tutoring.

2.3 Intelligent Tutoring Systems and Procedural Training

Intelligent Tutoring Systems (ITS) provide the closest educational precedent for this work. Classic ITS research distinguishes expert knowledge, learner state, and tutoring feedback, and has shown that step-based computer tutors can be effective in structured learning domains [21]. In procedural settings, constraint-based and example-tracing tutors model expected actions and compare learner behaviour against authored task knowledge [2, 17]. These ideas directly inform the use of procedure packs and gates in this thesis.

The difference is the observation interface. Many ITSs operate inside a tutor-controlled environment where learner actions are already represented as formal events. The simulator used here is not a tutor-native interface. The runtime must infer procedure state from DCS-BIOS telemetry, cockpit screenshots, sequence history, and occasional manual confirmations. This makes evidence integration part of the tutoring problem itself.

LLMs add a further tension. They can produce fluent and flexible instructional dialogue, and recent educational discussions recognise their promise as well as their reliability risks [13]. For cockpit procedures, fluency is not enough. The contribution of this thesis is therefore not an open-ended LLM tutor, but a constrained tutoring runtime in which generated language is downstream of evidence collection, retrieval, procedure inference, and validation.

2.4 Simulator-Based Training and Instrumentation

Simulator-based aviation training is a mature area, and prior work has examined how simulation supports skill acquisition and transfer in flight-training contexts [11]. DCS World provides the high-fidelity simulation environment used in this thesis [7]. The relevant research issue for the present work is instrumentation: how the tutoring system observes what the learner and aircraft are doing without replacing the simulator with a custom tutor interface.

DCS-BIOS addresses part of this problem by exposing cockpit state through a structured telemetry bridge [6]. Compared with screen capture alone, telemetry gives deterministic access to many switch positions, indicator states, and numerical values. However, telemetry does not cover every piece of evidence that matters for the cold-start, especially display-page content and text. The thesis therefore treats simulator instrumentation as a multimodal problem: telemetry is used where it is reliable and direct, while vision is reserved for the cockpit evidence that telemetry does not export.

This separation also improves auditability. A logged help cycle can later show which state was read from DCS-BIOS, which facts came from the VLM, which rule made a step eligible or blocked, and which retrieved source supported the generated explanation. Instrumentation is thus not only an engineering requirement but a condition for evaluating the tutor's behaviour.

2.5 Rule-Based and State-Aware Tutoring

The tutoring runtime combines deterministic state tracking with model-generated language. Rule-based approaches to procedure tracking, including finite-state, production-rule, and constraint-oriented models, offer interpretability because they make task structure explicit. The procedure pack in this thesis follows that principle: steps, preconditions, completion gates, visual bindings, and error categories are authored as declarative configuration rather than hidden in prompts.

The limitation of a purely rule-based tutor is that authored feedback can become brittle when the learner asks for help in an unexpected state or needs an explanation rather than a binary correction. The runtime therefore delegates natural-language formulation to an LLM, but not the authority to decide the procedure state. Step inference, eligibility, evidence availability, and overlay allowlists remain under deterministic control.

This hybrid division is related to broader work on grounding language models in action feasibility and symbolic structure, such as SayCan-style grounding of language in robotic affordances [1]. The present thesis adapts that grounding intuition to simulator tutoring: before a generated response can be acted upon, it must be tied to evidence that the runtime can observe, validate, and replay.

2.6 Foundation Models and VLMs for Situated Assistance

Foundation models provide two capabilities that are attractive for situated tutoring: language models can produce flexible explanations, and vision-language models can extract information from cockpit images. Parameter-efficient fine-tuning methods, especially Low-Rank Adaptation (LoRA), make it practical to adapt large models to narrow tasks with limited trainable parameters [12, 16]. The model backbones and VLM literature used in this thesis provide the general technical context for this adaptation setting [4, 5, 15].

The thesis uses these capabilities in a restricted way. The VLM is not asked for free-form cockpit descriptions, and the LLM is not asked to invent the next procedural state. Instead, the VLM outputs structured JSON visual facts, and the help-generation model receives an evidence packet assembled by the runtime. This choice follows the same general motivation as retrieval-augmented generation: model output should be conditioned on explicit external evidence rather than parametric memory alone [14].

The cockpit-display task also resembles structured screen understanding. Research on GUI and screen-content understanding has shown that visual models can reason about fixed regions, labels, and symbolic layout [23]. Cockpit displays share that structured quality, but the tutoring setting adds stricter consequences for false positives. A VLM that incorrectly reports a critical display state as visible may cause the tutor to advance the learner at the wrong time. This is why ontology design and critical-false-positive analysis are treated as central methodological issues in this thesis rather than as secondary benchmark details.

2.7 Explainable and Grounded AI Assistance in AR/VR

The original proposal placed the tutor in an immersive Quest 3 cockpit experience, and the implemented system retains that evaluation direction through the VR pilot. Prior work on AR guidance for maintenance and procedural tasks shows the value of presenting instructions in the user's task context, often through spatially registered text, arrows, or animations [18]. Such systems demonstrate why in-context guidance is attractive: the learner can keep attention on the task environment rather than switching to external materials.

The system studied here differs from pre-authored AR guidance because its help content is generated at runtime from current evidence. That makes explanation and traceability central. Work on explainable AI and human-AI interaction emphasises that users and developers need to understand why a system produced a recommendation, especially in high-stakes or safety-sensitive settings [3, 9]. In this thesis, the explanation need is operationalised through evidence references: each recommended step, overlay target, or refusal to advance can be traced back to telemetry, a rule, a visual fact, or a retrieved source.

This grounding requirement also limits the claims made about VR. The pilot study in Chapter 6 shows that the runtime and instrumentation can be exercised with real participants in VR, but it does not establish controlled educational effectiveness. The VR contribution is therefore part of the evaluation framework and formative evidence, while the primary thesis claim remains the design and validation of the evidence-grounded tutoring runtime.

2.8 Positioning of This Work

The thesis sits at the intersection of the areas reviewed above, but its contribution is the particular integration rather than membership in any single area. From intelligent tutoring systems it takes the need for explicit task models and state-sensitive feedback. From simulator training it takes the need for instrumentation that observes real learner behaviour in a high-fidelity environment. From rule-based assistance it takes deterministic procedure gates. From foundation-model work it takes flexible language generation and domain-adapted visual extraction. From explainable and AR/VR assistance it takes the requirement that help remain inspectable in context.

The resulting system is an evidence-grounded procedural tutoring runtime. Its pro-

cedure pack defines what counts as valid state evidence; DCS-BIOS supplies telemetry where available; the visual fact ontology supplies structured cockpit display observations where telemetry is missing; retrieval supplies procedural sources; and the help cycle constrains model output through validation, allowlists, fallbacks, logging, and replay. This is the research position developed in the rest of the thesis.

The claim boundary is equally important. The technical chapters and benchmarks evaluate whether the architecture and visual fact pipeline can produce reliable, auditable evidence for tutoring. The formative VR pilot provides descriptive evidence about feasibility, completion, error patterns, and instrumentation with real participants. Larger controlled studies are still required before making claims about learning gains, workload reduction, retention, or transfer.

3 System Architecture

This chapter answers RQ1 at the architectural level: how can simulator telemetry, retrieved procedural knowledge, and visual observations be integrated so that tutoring outputs remain state-aware, auditable, and safe for procedural guidance? The central design decision is to treat foundation-model output as one stage in an evidence-constrained runtime rather than as the tutor itself. Every help response is produced from an explicit evidence packet, checked against procedure-pack rules, validated by a harness, mapped through an overlay allowlist, and recorded for replay.

The chapter first derives the design requirements from the evidence-grounding problem. It then presents the ports-and-adapters architecture, the live help cycle, the telemetry and procedure-gate evidence layers, the harness validation mechanism, the simulator adapter boundary, logging and replay infrastructure, and the runtime role of VLM-derived visual facts. Dataset construction and VLM fine-tuning methodology are deferred to Chapter 4; implementation-level details appear in Chapter 5.

3.1 Design Requirements

Evidence-grounded procedural tutoring imposes requirements that are different from a conventional checklist application or an unconstrained LLM chatbot. The runtime must observe state, reason over procedure order, explain help in language, and still prevent unsupported overlay actions from reaching the learner. Six requirements guide the architecture.

- **G1 – Live procedural state observation.** The tutor must observe learner progress through simulator state rather than through self-report alone. In the F/A-18C case, this means using DCS-BIOS telemetry where available and visual facts where cockpit display state is not exported as structured data.
- **G2 – Procedure-aware step inference.** The tutor must reason over preconditions, completion gates, and sequence history. The next useful help item is not

necessarily the next checklist line; it is the next action that is valid given the current evidence.

- **G3 – Explicit evidence for actionable output.** Every recommended step, overlay target, or refusal to advance must be traceable to evidence: telemetry, a gate result, a retrieved procedural source, a recent state change, or a visual fact. Model fluency alone is not acceptable evidence.
- **G4 – Clean architectural boundaries.** Domain logic must remain independent of simulator APIs, model providers, GPU availability, and VR display plumbing. This permits the core procedure and validation logic to be tested without a running simulator.
- **G5 – Graceful degradation.** When model output is unavailable, malformed, or insufficiently grounded, the runtime must fall back to deterministic text guidance or safe rejection rather than executing an unsafe overlay.
- **G6 – Declarative procedure configuration.** Procedure steps, gates, telemetry mappings, UI targets, visual fact bindings, and error taxonomies must be represented as a procedure pack rather than hidden inside prompts or adapter code. This keeps the task model inspectable and versionable.

These requirements explain why the system is designed as an evidence pipeline. The architecture does not try to make the language model internally reliable enough to act alone. Instead, it surrounds generation with explicit state evidence, declarative rules, schema validation, allowlists, and replayable logs.

3.2 Ports-and-Adapters Architecture

The runtime follows a ports-and-adapters architecture with a strict dependency direction: external systems connect through adapters, adapters implement stable ports, and the core contains the procedure and validation logic. The core never imports DCS-specific, provider-specific, or VR-specific modules. This boundary is central to the thesis claim because evidence-grounding must be testable and inspectable without depending on the live simulator.

Core layer. The core contains simulator-independent domain logic: procedure state tracking, gate evaluation, visual fact state merging, BM25 retrieval primitives,

event types, response schema generation, evidence packet construction, harness validation, scoring, and replay-oriented state checks.

Ports layer. Ports define the stable interfaces between the domain core and the outside world. The main ports cover model inference, knowledge retrieval, visual input, and telemetry input. These ports make it possible to replace a model provider, knowledge backend, or simulator adapter without changing the procedure engine.

Adapters layer. Adapters contain external integration code: DCS-BIOS receivers, telemetry sanitisation and aggregation, DCS overlay dispatch, OpenAI-compatible and local model clients, BM25 document retrieval, screenshot capture, and VLM visual fact extraction.

Procedure packs. The procedure pack is the task-specific contract consumed by the core. The F/A-18C pack defines step order, precondition gates, completion gates, UI target identifiers, telemetry variable mappings, visual fact bindings, and error taxonomy. It is not an adapter because it does not talk to the simulator; it is the declarative domain specification.

Figure 3.1 summarises this architecture. The diagram should be read from left to right as an evidence flow: simulator state and cockpit images enter through adapters, procedure-pack rules structure the runtime interpretation, retrieved sources and model services support help generation, and the harness checks the resulting action plan before any overlay or tutor text reaches the cockpit.

The same boundary also supports the research workflow. Technical claims about gating, evidence packets, harness decisions, and replay can be tested with fixtures. Claims about visual fact extraction can be benchmarked separately from live tutoring. Claims about VR presentation can be treated as formative pilot evidence rather than as a prerequisite for validating the architecture.

3.3 The Evidence-Grounded Help Cycle

The live help cycle is the core runtime calling chain. In compact form, it is:

help trigger → telemetry snapshot → optional vision capture → step and gate inference → evidence packet → retrieval → model response → harness validation → overlay/text output → event log

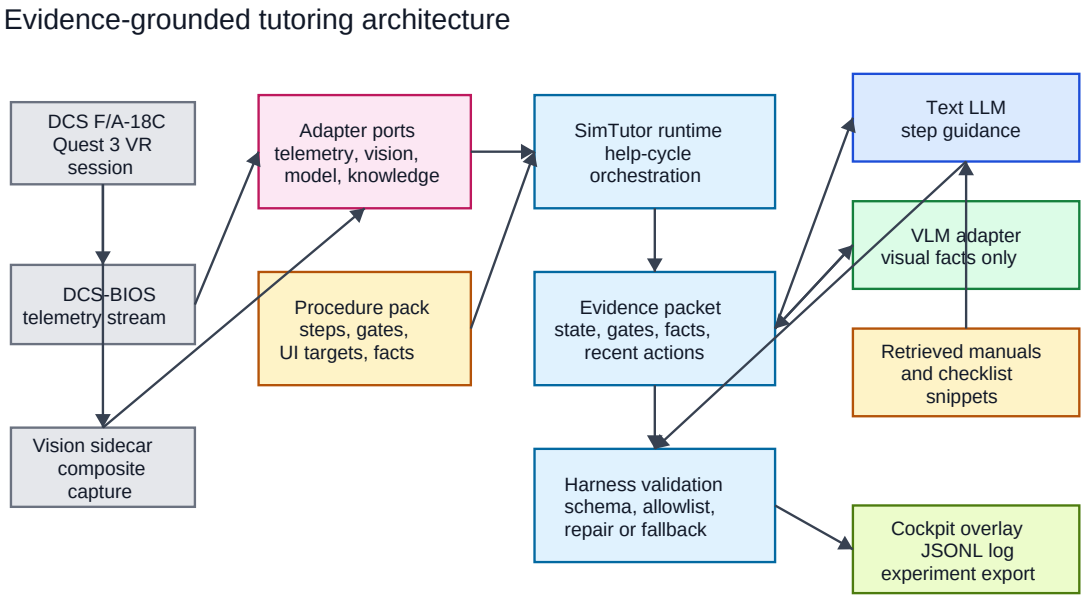

Figure 3.1: Evidence-grounded tutoring architecture. Simulator telemetry, composite-panel vision, procedure-pack configuration, retrieved knowledge, and model outputs are integrated through an evidence packet and checked by the harness before any overlay or tutor text reaches the cockpit.

This sequence matters because it fixes the order of authority. The model does not first decide what the simulator state means. Instead, the runtime assembles state evidence and candidate steps before generation, and validates the generated response before display. Figure 3.2 shows this cycle.

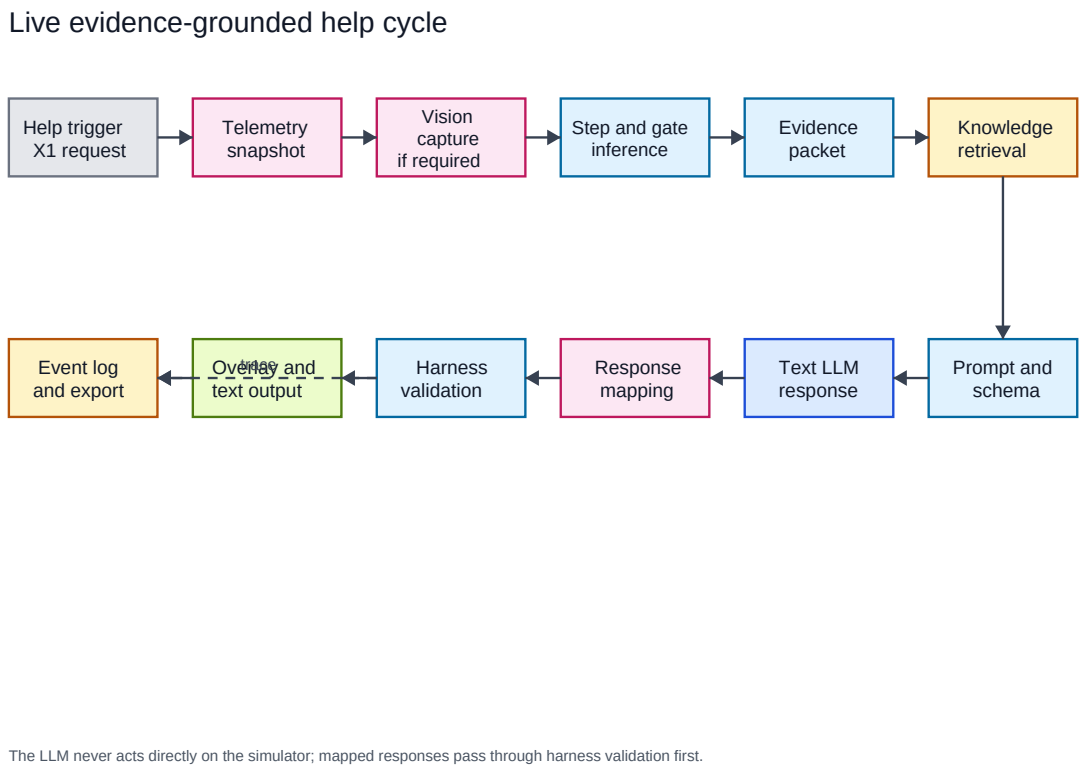

Figure 3.2: Live evidence-grounded help cycle. Each help request is converted into a telemetry and vision observation, interpreted against procedure gates, grounded with retrieved knowledge, passed through the text LLM, validated by the harness, and then dispatched as overlay/text output with an event-log trace.

3.3.1 Help Cycle Stages

Observation. The DCS-BIOS adapter receives simulator state and the telemetry pipeline normalises it into runtime variables. When the active or likely step requires display evidence, the vision adapter can capture the composite cockpit panel and request visual fact extraction.

Candidate inference. The procedure engine evaluates precondition and completion gates, identifies candidate steps, and records missing or uncertain evidence. The result-

ing candidates are not natural-language suggestions; they are structured runtime objects derived from the current evidence.

Evidence packet assembly. The runtime collects telemetry values, gate results, recent deltas, visual fact states, retrieved source references, and step candidates into an evidence packet. This packet is the boundary object between state interpretation and model-assisted help generation.

Model response. The prompt asks the model to produce a structured help response grounded in the packet. The output must conform to the runtime HelpResponse schema. Free-form text that cannot be parsed or mapped safely does not become an action.

Validation and mapping. The response passes through JSON extraction, schema validation, evidence reference checks, step and target allowlists, and harness validation. Only then can a response be mapped to overlay actions or tutor text. Failed validation triggers repair, rejection, or deterministic fallback.

Execution and logging. Overlay commands are limited to safe display actions such as highlight, pulse, or clear. Control injection and auto-clicking are outside the architecture. Every help cycle is logged as structured events so that the decision can be replayed after the session.

3.3.2 Schema-Level Evidence Grounding

Evidence grounding is enforced before output execution. The HelpResponse schema is generated from the current procedure pack and UI map, so the set of valid steps and overlay targets is task-specific. Each target proposed by the model must be accompanied by evidence whose type and reference correspond to data available in the help-cycle context. The supported evidence kinds are telemetry variables (`var`), gate evaluations (`gate`), retrieved sources (`rag`), recent state changes (`delta`), and visual facts (`visual`).

This design changes the failure mode of an unsupported model response. If the model proposes a plausible but ungrounded overlay, the response is not merely logged as questionable; it is structurally invalid for action execution. The system can then repair to an allowed target, fall back to text-only guidance, or reject the action plan.

3.4 Telemetry as State Evidence

Telemetry is the primary evidence source for controls, indicators, and numerical state values that DCS-BIOS exports directly. The telemetry pipeline turns raw simulator

frames into stable state evidence through four stages:

1. **Raw ingestion.** DCS-BIOS UDP frames are decoded into structured observations with timestamps and sequence information.
2. **Variable resolution.** Procedure-pack mappings project raw BIOS fields onto normalised variables such as `rpm_r`, `apu_ready`, or `battery_on`. The resolver also tracks missing sources so the runtime can distinguish an observed false value from an unavailable value.
3. **Delta sanitisation.** Configured thresholds, debounce windows, and filtering policies suppress spurious changes from sensor jitter, switch bounce, and out-of-order frames.
4. **Delta aggregation.** Recent state changes are compressed into a prompt-safe summary and an evidence source for help-cycle reasoning.

The missing-source distinction is important for procedural tutoring. A gate should not fail in the same way when a switch is known to be off and when the telemetry source is absent. The first case can justify corrective guidance; the second may require uncertainty-aware or text-only help.

3.5 Procedure Engine and Evidence Gates

The procedure engine maintains state over the ordered cold-start sequence. Each step can be pending, active, done, or blocked, and transitions are driven by precondition and completion gates. Gates are expressed declaratively in the procedure pack rather than embedded in prompts. This ensures that procedure state remains a rule-governed domain decision rather than a model inference.

The gate evaluator supports threshold checks, range checks, boolean flags, and elapsed-time conditions. Missing, source-missing, or sentinel values produce diagnostic failures instead of being silently coerced into valid state. These diagnostics feed both the evidence packet and deterministic fallback.

3.5.1 Deterministic Fallback

The deterministic fallback is part of the architecture, not an emergency afterthought. When no model is available or when model output fails validation, the runtime computes

a conservative hint from the active step, unsatisfied gates, recent actions, and evidence availability. It distinguishes hard blockers, where evidence shows that a gate is unsatisfied, from soft blockers, where the required evidence is missing or uncertain. Overlay output is suppressed unless a local rule can uniquely determine a valid target.

3.5.2 Step Classification by Evidence Source

The current F/A-18C procedure pack classifies steps according to the most reliable available evidence source:

- **Vision-priority steps:** S08, S15, S18, and S19 require cockpit display evidence from the visual fact pipeline.
- **BIOS-priority steps:** S01, S03–S13, S16, and S20–S33 are declared as primarily telemetry-verifiable in the pack metadata.
- **Manual or out-of-layout steps:** S14 and S17 require manual confirmation or involve cockpit areas outside the three-display composite.

The fire-test step S02 is handled through its own gate and recent-action contract rather than through the pack's three priority lists. The important architectural point is that the runtime does not ask the model to decide which evidence channel is authoritative for a step. That choice is declared by the procedure pack and consumed by the help cycle.

3.6 Harness Engineering and Evidence Validation

The harness is the architecture's final safety boundary before output reaches the simulator. It converts a model-proposed response into a validated action plan, or into a repaired, rejected, or text-only outcome. The detailed object relationships are shown in Appendix Figure 9.1; the main architectural role is summarised here.

The harness uses four conceptual objects:

`EvidencePacket`. A normalised snapshot of the current help cycle: telemetry state, gate diagnostics, recent actions, visual facts, retrieved references, and candidate step information.

`StepCandidate`. A candidate explanation of where the learner may be in the procedure, derived from gates, telemetry, recent actions, and visual evidence.

`StepHarnessSpec`. A pack-derived contract for each step: allowed overlay targets, required observables, telemetry facts, vision facts, completion predicates, recovery policy, and signal freshness requirements.

`HarnessActionPlan`. The validated result: selected step, selected overlay step, targets, evidence references, guidance text, text-only status, repair status, rejection status, and reasons.

The validation sequence has three effects. First, it prevents wrong-step or wrong-target outputs from bypassing the procedure pack. Second, it prevents stale or inappropriate evidence from driving a later step. Third, it creates machine-readable reasons for repairs and rejections, which are then available for replay and evaluation.

3.6.1 Fixture-Based Regression Testing

The harness is tested with live help fixtures: recorded telemetry snapshots and runtime contexts captured at procedure states that previously exposed failures. Each fixture is replayed through the help cycle and asserts the expected outcome, such as the correct target, a repaired target, text-only fallback, or safe rejection.

The current coverage matrix contains 20 live-fixture regression entries across S01–S33. These fixtures cover correct advancement, wrong-target rejection, repair of incomplete or mismatched visual evidence, prevention of stale VLM facts leaking into non-visual steps, moving-control wait guidance, and interaction semantics for switches, push buttons, and rotary controls. This coverage is not an educational evaluation; it is technical evidence that evidence-grounding constraints can be enforced systematically at the runtime boundary.

3.7 Adapter Boundary: Simulator, Overlay, and Vision

The adapter layer isolates DCS-specific behaviour behind the ports used by the core. Three adapter responsibilities are central to the live runtime.

Telemetry input. The telemetry adapter receives DCS-BIOS state data and feeds the telemetry pipeline. Its output is normalised state evidence rather than simulator-specific frame data.

Overlay output. The overlay adapter maps abstract tutor targets to DCS overlay identifiers and sends highlight, pulse, or clear commands. It does not execute cockpit controls.

Vision input. The vision adapter triggers screenshot capture and passes composite-panel images to the VLM fact extractor. The resulting visual facts return to the core as structured evidence.

The VR path follows the same boundary. VR mirror display, tutor text transport, and composite-panel capture are adapter concerns; procedure inference and harness validation remain in the core. The VR code path was exercised in the formative pilot study reported in Section 6.3. Larger controlled human-subject evaluation remains future work.

3.8 Event Logging and Replay

Auditability requires that help-cycle decisions survive beyond the live session. The runtime records versioned, timestamped events in JSONL format. Events include observations, help requests, evidence summaries, model outputs, harness decisions, overlay actions, and scoring-relevant interaction data.

Replay uses these records to reconstruct step ordering, state-machine consistency, and help-cycle behaviour offline. The current replay evaluation suite includes five scenario types: no-operation, battery-on, battery-and-generator-off, FCS reset and BIT, and INS alignment. These cases use DCS-BIOS raw captures and, where available, vision sidecar artifacts.

The logging and replay layer supports the thesis in two ways. During development, it makes regressions reproducible. During evaluation, it connects claims about readiness and instrumentation to concrete artifacts rather than to manual observation alone. Full end-to-end replay of multimodal human-in-the-loop sessions with synchronised vision frames remains a limitation.

3.9 VLM Adapter for Visual Evidence

The VLM component supplies visual evidence for display states that telemetry does not export directly. Its runtime output is a set of visual facts, not a procedure decision. Each fact is a bounded proposition about visible cockpit display content, with state

`seen`, `not_seen`, or `uncertain`. The procedure engine and harness decide how those facts affect the help cycle.

The current runtime contract uses the 13-fact ontology introduced in Chapter 2. Facts have time-to-live settings, optional sticky semantics, and step bindings through all-of or any-of conditions. For example, facts related to FCS-MC final GO can be latched so that a transient completed display state is not lost when the learner changes pages.

When a help request concerns a vision-priority step, the runtime may trigger a capture, extract visual facts, merge them into the current evidence snapshot, and include them in the evidence packet. If the vision adapter is unavailable or the multimodal request fails, the help cycle continues with telemetry, retrieval, and procedure gates. This graceful degradation preserves the telemetry-grounded core while allowing visual evidence to improve steps that need display interpretation.

The data pipeline, ontology evolution from 8 facts to 13 facts, LoRA fine-tuning configuration, and holdout benchmark protocol are methodological questions rather than architecture questions. They are therefore treated in Chapter 4 and evaluated in Chapter 6.

3.10 Summary

The architecture presented in this chapter turns foundation-model tutoring into a constrained evidence-processing pipeline. Telemetry and visual facts provide observations, procedure packs define valid state transitions and evidence requirements, retrieval supplies source-grounded context, models generate structured help, the harness validates or repairs proposed actions, and logs make the decision replayable. This is the system-level answer to RQ1: procedural help becomes auditable when model output is made downstream of explicit evidence and upstream of enforced validation.

4 Methodology

Chapter 3 presented the runtime architecture that keeps tutor output downstream of evidence and upstream of validation. This chapter describes how that evidence is produced, curated, and evaluated. The method is organised around a practical question: how can simulator telemetry, retrieved procedure knowledge, and VLM-derived visual facts become auditable inputs for a procedural tutor?

The chapter has three roles. First, it defines the runtime evidence interfaces used by the procedure engine: telemetry variables, gating diagnostics, retrieved snippets, and deterministic fallback. Second, it presents the visual fact methodology for RQ2: screenshot capture, pre-labelling, human review, ontology revision, bilingual SFT export, LoRA adaptation, and holdout benchmarking. Third, it documents the formative VR pilot protocol for RQ3. The pilot is treated as descriptive evidence about feasibility and interaction patterns; it is not used for inferential claims about learning gains.

4.1 Procedure Runtime and Telemetry Grounding

Telemetry and procedure gates provide the method-level state evidence on which the help cycle depends. The architectural placement of this layer was described in Chapter 3; the methodological concern here is how raw simulator observations become stable, inspectable variables for later benchmarking, replay, and help-cycle analysis.

4.1.1 Variable Resolution and Source-Missing Tracking

The procedure pack declares a telemetry map that projects raw DCS-BIOS fields onto normalised runtime variables. The resolver produces a flat dictionary of key–value pairs and separately tracks unavailable sources in `vars_source_missing`. This separation is important because a false value and an unobserved value should not have the same methodological meaning. A switch known to be off can support corrective guidance. A missing source supports only uncertainty-aware guidance or fallback.

4.1.2 Gating Rule Evaluation

Each procedure step contains precondition and completion gates. The gate DSL supports threshold checks, range checks, boolean flags, and elapsed-time conditions. Missing, source-missing, or sentinel values fail with diagnostic reasons rather than being silently coerced. The failed gate and reason are included in the help-cycle context and can later be inspected in logs. This turns procedure progress into a rule-governed evidence claim rather than a model-inferred narrative.

4.1.3 Delta Sanitisation and Aggregation

Raw telemetry can change at high frequency because of simulator physics, instrument jitter, and transitional switch states. Delta sanitisation applies per-variable thresholds, debounce windows, and filtering rules. Delta aggregation then compresses recent, sanitised changes into a prompt-safe summary. The resulting deltas are used as evidence of recent learner action without exposing every raw simulator frame to the language model.

4.1.4 Deterministic Fallback

Fallback is part of the method rather than only an implementation safeguard. When model output is unavailable or invalid, the runtime computes a conservative hint from the active step, unsatisfied gates, recent actions, and evidence availability. The fallback distinguishes hard blockers, where observed evidence shows that a gate is unsatisfied, from soft blockers, where required evidence is missing. Overlay guidance is suppressed unless a local rule identifies a unique valid target.

4.2 Knowledge Grounding and Source Policy

Retrieved procedural knowledge supplies textual context for generated help, but it is deliberately source-controlled. The local retrieval adapter indexes curated procedure and aircraft documentation at sentence level and ranks snippets with BM25 [19]. A grounding query is constructed from the active step, its description, and the current observation; the top snippets are included in the help-cycle context.

A declarative source policy restricts retrieval to allowed document sources. This prevents the model from receiving procedure variants outside the active procedure pack, such as carrier-specific instructions when the current profile is an airfield cold start. In

the thesis method, retrieval is therefore not a general document search facility. It is a controlled evidence channel whose scope is part of the procedure configuration.

4.3 Visual Fact Data Pipeline

The visual fact pipeline is the main methodology for RQ2. It turns DCS cockpit screenshots into supervised examples and then into a benchmarked visual fact extractor. The pipeline is designed for traceability: a benchmark error should be traceable back to the model prediction, the human-reviewed label, the pre-label, and the original image.

Figure 4.1 shows the pipeline as a loop. Holdout failures are not only final results; they can reveal weaknesses in the ontology and guide later data collection.

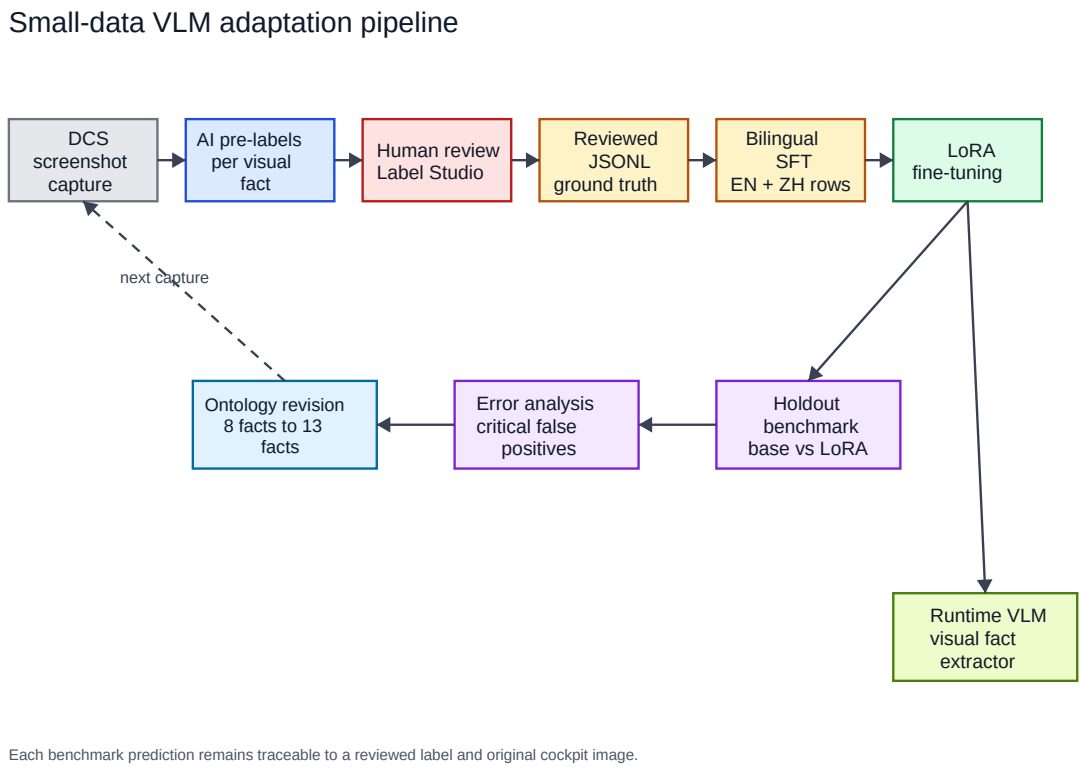

Figure 4.1: VLM adaptation pipeline. DCS composite screenshots are pre-labelled, human-reviewed, converted into bilingual supervised fine-tuning rows, adapted with LoRA, evaluated on independent holdout sets, and used to drive ontology revision.

1. **Screenshot capture.** DCS renders the left DDI, AMPCD, and right DDI into one composite-panel image. The same composite layout is used for data collection,

pre-labelling, training, benchmarking, and runtime inference. This keeps the input distribution stable across the method.

2. **VLM pre-labelling.** A large external VLM produces initial labels and evidence notes for each image. Pre-labelling is used as an annotation aid, not as ground truth.

3. **Human review.** Pre-labelled samples are reviewed in Label Studio [20]. The reviewer corrects fact states and evidence notes while preserving a link to the raw image and pre-label.

4. **Reviewed JSONL export.** The reviewed export is the authoritative ground-truth artifact for downstream training and evaluation.

5. **Bilingual SFT generation.** Each reviewed image yields an English and a Chinese training row with the same image and fact labels. The assistant target is structured JSON containing visual facts and a short summary; external management fields such as frame identifiers, file paths, and confidence fields are excluded from the target.

6. **LoRA adaptation.** The SFT rows are used to train LoRA adapters for VLM backbones [12]. The implementation uses PEFT adapter definitions [16], TRL supervised fine-tuning infrastructure [22], and Unsloth acceleration for 4-bit VLM training [10].

7. **Holdout benchmark and revision.** Base and LoRA configurations are evaluated on independent holdout sets. Errors are inspected at fact level, especially for completion-critical facts, and the resulting failure modes feed back into ontology and dataset revisions.

The method intentionally keeps the VLM's task narrow. The model is trained to report visual facts with states `seen`, `not_seen`, or `uncertain`. It is not trained to decide procedure completion or produce tutor actions. Those decisions remain in the procedure engine and harness.

4.4 Dataset Curation

4.4.1 Dataset Overview

Six reviewed datasets support the visual fact work across two ontology generations. Table 4.1 lists their roles and tracked counts.

Table 4.1: Dataset inventory used for visual fact extraction.

Run	Dataset	Samples	Facts	Lang.	Role
Run-001	v1 SFT	180	8	en/zh	v1 training source
Run-002	v1 holdout	50	8	en	v1 holdout
Run-002	v2 newfacts holdout	50	13	en	v2 holdout
Run-003	v2 SFT	220	13	en/zh	v2 training source
Run-004	v2 random holdout	100	13	en	v2 holdout
Run-005	composition rebalance	122	13	en/zh	v2 training supplement

Run-003 and Run-005 contain samples whose free-text summary field is blank (82 and 25 samples respectively). These samples remain valid for the visual fact task because every sample still contains a complete fact array. The supervised target used by the runtime and benchmark is the structured fact state, not the optional summary.

4.4.2 Ontology Evolution: 8-Fact v1 to 13-Fact v2

The first ontology contained eight facts over the composite cockpit panel. It established that a small reviewed dataset could improve structured cockpit fact extraction, but it also exposed a representation problem: some targets mixed visual observation with procedural interpretation. The clearest case was `ins_go`, which asked a single-frame VLM to infer an INS completion state rather than report directly visible evidence.

Figure 4.2 illustrates the methodological correction. Compound procedural targets were decomposed into locally observable visual atoms, leaving temporal and procedural interpretation to the tutor runtime.

The current 13-fact ontology follows three design rules:

1. **Decompose compound targets.** The `ins_go` target is replaced by separate visual atoms for GRND alignment text and INS OK text. The FCS-MC sequence is represented by page, intermediate-result, in-test, and final-GO facts.

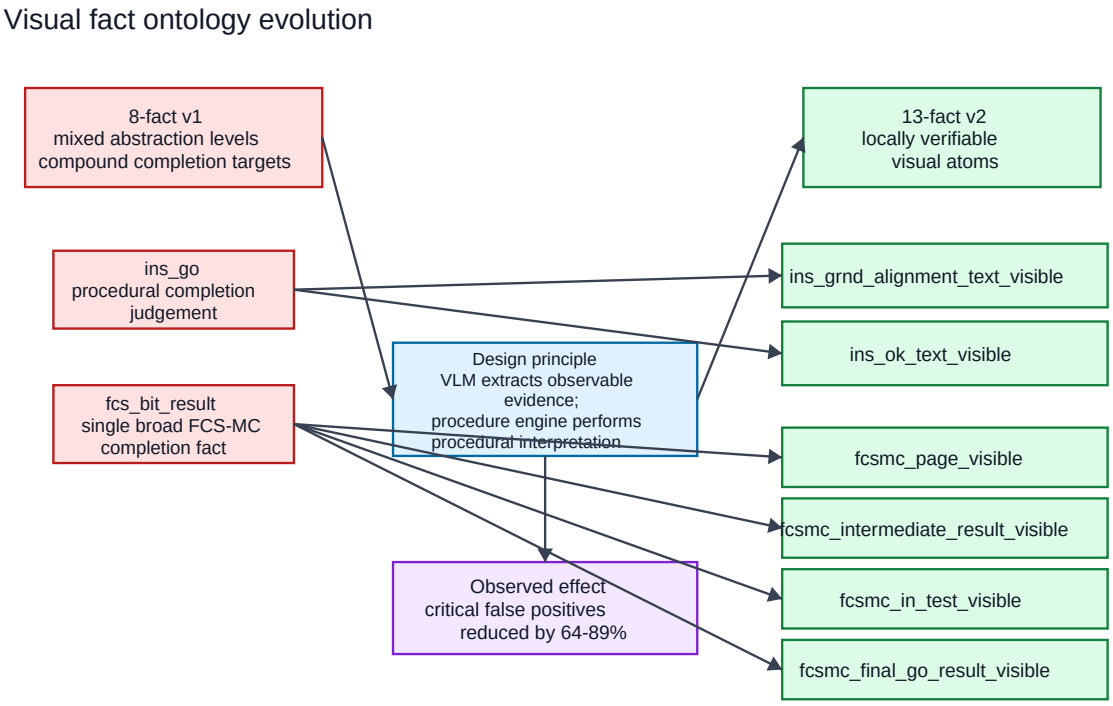

Figure 4.2: Ontology evolution from 8-fact v1 to 13-fact v2. Compound targets such as `ins_go` and broad FCS-MC completion facts were decomposed into locally verifiable visual atoms; the procedure engine then combines those observations with telemetry, phase, and timing evidence.

2. **Separate region from semantics.** Fact names describe the visible state rather than encoding the display region in the identifier. Region constraints are part of the ontology metadata.

3. **Keep facts locally observable.** Each fact should be decidable from the composite image without requiring procedure history. The procedure engine combines facts with telemetry, step bindings, and timing information later.

This ontology discipline is central to the thesis. It defines the boundary between perception and procedural reasoning: the VLM reports visual evidence, while the runtime decides whether that evidence satisfies a procedural step.

4.4.3 Training Recipe: Run-003 + Run-005×2

The primary v2 training recipe combines Run-003 with a repeated Run-005 composition-rebalance supplement. Run-003 contributes 220 reviewed images, exported bilingually as 440 SFT rows. Run-005 contributes 122 reviewed images, exported bilingually as 244 rows; this supplement is included twice, yielding 488 rows. The final recipe therefore contains **928 SFT rows**.

Run-005 targets under-represented positive states from Run-003, including INS OK text, FCS page X marks, and FCS-MC sub-state facts. The repeated supplement is a deliberate composition rebalance strategy, not an independent validation set. For that reason, generalisation claims are based on external holdouts rather than on an internal same-pool evaluation split.

4.4.4 Holdout Independence

Two 13-fact v2 holdouts are used for the main evaluation. Run-002 newfacts contains 50 images from an earlier capture session re-labelled under the 13-fact ontology. Run-004 random contains 100 randomly sampled cockpit-state captures. Local overlap checks report zero exact overlap between either holdout and the Run-003 + Run-005 training union.

The earlier Run-001 evaluation line is treated differently. Because it overlaps with the training data pool, it is useful as a contaminated development benchmark for checking schema learning and historical ontology behaviour, but it is not used as independent evidence for v2 generalisation.

4.5 LoRA Fine-Tuning Configuration

4.5.1 Backbone Models and Comparison Strategy

The primary adaptation line uses `Qwen/Qwen3.5-9B-Base` with the Run-003 + Run-005×2 recipe. Two supplementary backbones are included to test whether the same data recipe remains meaningful under scale, version, and family changes: `Qwen/Qwen3.6-27B` and `google/gemma-4-31B`. These comparisons are methodological controls rather than separate thesis claims. Chapter 6 reports the numerical results.

The comparison holds the main factors constant wherever possible: ontology, training recipe, benchmark prompt, holdout sets, generation temperature, output schema, and metric computation. Backbone-specific differences remain and are therefore reported explicitly. Qwen-family runs apply LoRA to vision and language modules; the Gemma run uses an all-linear LoRA configuration with vision layers not separately fine-tuned. Full settings appear in Appendix 9, Table 9.1.

4.5.2 Training Stack and Hyperparameters

All backbone runs use 4-bit loading, LoRA rank $r = 16$, $\alpha = 16$, dropout 0.0, four epochs, learning rate 2×10^{-4}, effective batch size 4, maximum sequence length 4096, seed 3407, warmup ratio 0.1, and weight decay 0.01. Adapter weights are saved separately from the base model.

The epoch count and LoRA rank are chosen for small-data domain adaptation. With 928 SFT rows, the adapter must learn both the fixed JSON schema and the visual boundaries of cockpit facts. A very small number of epochs may not teach those boundaries reliably, while excessive training would risk memorising the capture sessions. Rank 16 provides moderate adaptation capacity without turning the adapter into a high-capacity task-specific model.

4.5.3 Bilingual SFT and Composition Rebalance

The bilingual SFT strategy creates two instruction variants per reviewed image: one English and one Chinese. The image and fact labels remain identical. This increases instruction-format coverage without requiring additional image annotation. It does not replace the need for new cockpit states or hard negative examples.

Composition rebalance is handled through Run-005. Because rare positive facts are especially important for downstream procedure reasoning, the training recipe increases

their exposure by adding targeted captures and repeating the Run-005 bilingual rows. The method therefore separates two questions: whether the adapter can learn the ontology from the rebalanced training recipe, and whether the learned extractor generalises to independent holdouts.

4.6 Benchmark Protocol

4.6.1 Metrics

The benchmark compares model predictions against human-reviewed ground truth at both sample and fact level. The metrics are selected to reflect downstream runtime risk:

- `json_valid_rate`: proportion of samples with parseable JSON output.
- `schema_valid_rate`: proportion of samples whose JSON contains the required fact identifiers exactly once and uses only valid states.
- `fact_accuracy`: three-class accuracy over all facts and samples.
- `macro_f1`: macro-averaged F1 over the three states, averaged over facts.
- `seen_f1`: F1 for the positive `seen` class, averaged over facts. This matters because missed or hallucinated visible states change procedure evidence.
- `sample_exact_match`: proportion of samples where all 13 facts match the reviewed labels.
- `critical_false_positive_count`: count of completion-critical facts predicted `seen` when the reviewed label is `not_seen` or `uncertain`.

Critical false positives are tracked separately because they are the most dangerous visual error for the tutor. A false positive on a completion-critical fact could make the procedure engine believe that a display state has been observed when it has not.

4.6.2 Base vs. LoRA Comparison

Base and LoRA configurations receive the same prompt, images, generation parameters, and output schema. Decoding uses temperature 0.0, maximum new tokens 1024, and seed 3407. Predictions are written to JSONL, parsed, normalised, and scored without post-hoc filtering. The comparison artifact reports LoRA-minus- base deltas, while Chapter 6 interprets those deltas.

4.6.3 Benchmark Categories

Benchmark runs are classified by evidence status. Contaminated development sets come from the same data pool as training and are used only for regression and historical interpretation. Held-out new-session benchmarks use independent captures with zero exact training overlap. This distinction determines the strength of the claim that can be made from a metric.

4.6.4 Benchmark Artifacts

Each benchmark run produces a standard directory with metrics for each model, per-sample predictions, error JSONL files, per-fact score CSVs, comparison summaries, human-readable reports, and charts. These artifacts form the evidence chain for the technical results reported in Chapter 6.

4.7 Human-Subject Pilot Methodology

A small formative VR pilot study was conducted to gather descriptive evidence about the tutor in use during the F/A-18C cold-start task. The pilot was not a powered controlled experiment. Its purpose was to exercise the runtime in a realistic VR setting, inspect interaction logs, and identify procedural error patterns for later study design.

4.7.1 Participants and Conditions

Nine participants (P01–P09) completed one trial each. The study used a between-subjects design with non-random assignment based on scheduling availability:

With tutor (n=4). P04, P05, P08, and P09 received overlay highlights, structured help text, telemetry-grounded step inference, and VLM visual fact support where applicable.

Without tutor (n=5). P01, P02, P03, P06, and P07 received no tutor assistance. Telemetry was still recorded for offline step inference and error coding.

Participant experience profiles are reported in Appendix 9, Table 9.5. Because assignment was not randomised and the sample is small, condition comparisons are descriptive only.

4.7.2 Task and Apparatus

All trials used the 33-step F/A-18C cold-start procedure in DCS World on a Meta Quest 3 via Quest Link. The composite cockpit panel was mirrored for experimenter observation. Tutor-condition trials used the live runtime with remote model inference through an OpenAI-compatible endpoint. A trial ended when the participant completed the procedure or voluntarily stopped after being unable to progress.

4.7.3 Data Sources and Coding

The export pipeline produced per-trial summaries, per-step coding files, help-cycle logs for tutor trials, action timelines, quality-gate metadata, and session summaries. Step errors were coded with five categories: omission (OM), commission (CO), order error (OR), prompting/assistance error (PA), and state violation (SV). The automated coding output was reviewed manually using telemetry, help-cycle logs, and action timelines; the human-coded columns are the authoritative source for reported pilot results.

4.7.4 Data-Quality Notes

Two trial-level caveats affect interpretation. First, P08's automatic summary reported incomplete progress, but manual telemetry review confirmed completion of all 33 steps. The manual coding is used for analysis, and the pre-review file is preserved for audit. Second, P02's questionnaire metadata contains an internal participant-id mismatch. P02's task metrics are retained, but P02 is excluded from questionnaire-derived workload or confidence summaries.

4.7.5 Analysis Approach

The pilot analysis reports per-participant outcomes and condition-level descriptive summaries. No null-hypothesis significance tests are performed. Observed durations are documented but not interpreted as completion-time comparisons because incomplete without-tutor trials ended at different stopping points. The analysis focuses on completion status, manual error categories, and tutor interaction patterns such as help requests, VLM calls, overlay actions, and fallback events.

4.8 Summary

The methodology establishes the evidence chain used by the rest of the thesis. Telemetry and retrieval provide controlled runtime context. The visual fact pipeline produces a reviewed and benchmarked perception layer for RQ2. The 13-fact ontology separates perception from procedural reasoning. The LoRA protocol uses matched data and holdouts to evaluate small-data adaptation without relying on contaminated development sets. The pilot protocol provides formative VR evidence for RQ3 while keeping its claims descriptive. Together, these methods make the later results auditable without overstating what the available evidence can prove.

5 Implementation

Chapters 3 and 4 described the architecture and methods of the evidence-grounded tutor. This chapter explains how those ideas are implemented as a running research artifact. The emphasis is selective: implementation details are included when they enforce an evidence boundary, preserve a Clean/Hexagonal separation, make a help cycle auditable, or support the evaluation artifacts used in Chapter 6.

The implementation question for the chapter is therefore not simply which modules exist, but where authority is checked before generated help reaches the learner. The answer is distributed across pack-derived schemas, evidence packets, response mapping, harness validation, bounded overlay execution, fallback logic, and append-only event logs. Operational details that do not change this argument, such as full command references and complete deployment topology, are kept in Appendix 9.

5.1 Runtime Boundary and Configuration

The codebase is organised around a Clean/Hexagonal boundary. Domain logic lives in `core/`; stable interfaces are declared in `ports/`; external systems are connected through `adapters/`; and task-specific procedure knowledge is stored in the F/A-18C procedure pack. This layout is an implementation decision, not only a code style preference. It keeps simulator transport, model providers, visual capture, retrieval, overlay dispatch, and experiment export outside the domain objects that decide what evidence is admissible.

The Python project targets Python 3.10+ and uses the dependency set declared in `pyproject.toml`. The runtime depends on a small number of general libraries: `httpx` for HTTP model calls, `PyYAML` for declarative configuration, `jsonschema` for contract validation, and `Pillow` for image handling in the VLM path. Tests are implemented with `pytest`. These stack choices matter mainly because they support inspectable file-based configuration and reproducible validation.

Model access is intentionally provider-agnostic. The text-help path uses an OpenAI-

compatible adapter and can be configured for servers such as vLLM, llama.cpp, or TGI. Local compatibility is provided through an Ollama adapter, and deterministic stubs support tests. The same boundary is used for the multimodal VLM path: the runtime treats visual fact extraction as an adapter that returns structured observations, not as a source of procedural authority.

The DCS-side integration has a similarly narrow role. Lua scripts and sidecar components export DCS-BIOS state, trigger composite-panel capture, and apply overlay highlight commands. They do not decide the active procedure step, do not generate tutoring text, and do not bypass the Python-side validation chain. This keeps the simulator integration as an I/O adapter while the procedure pack, evidence packet, and harness remain the sources of tutoring authority.

5.2 Evidence Grounding Enforcement

Evidence grounding is enforced through several implementation gates, not only through prompt wording. The model may propose a diagnosis, next step, explanation, and overlay target, but that proposal must survive contracts generated from the procedure pack and checks against the evidence available in the current help cycle.

Pack-derived schemas. The function `build_help_response_schema` generates the `HelpResponse` JSON Schema at runtime. Step identifiers come from the step registry, overlay targets come from the UI map, and error categories come from the taxonomy. The schema requires each overlay target to have at least one corresponding evidence item. Evidence items must use one of the runtime grounding types: `var`, `gate`, `rag`, `delta`, or `visual`. This makes the output contract procedure-specific rather than a generic language-model format.

Evidence packet construction. The help prompt is built from a structured context rather than from raw log fragments. `build_evidence_packet` aggregates telemetry evidence, gate evidence, recent actions, visual facts, deterministic candidates, and known conflicts into a compact packet. `build_help_prompt_result` then includes this packet, the allowed evidence references, retrieved snippets, and the overlay target policy in the model-facing prompt. As a result, an evidence reference can later be checked against the same context that was shown to the model.

Model output mapping and harness validation. The response path is deliberately narrow. JSON extraction removes code fences and reads the first valid object, but it does not repair arbitrary malformed text into a new answer. The parsed object is validated against the dynamic schema and then passed to the response mapper. The mapper checks that overlay evidence references are present in the help-cycle context, verifies overlay targets against allowlists, records mapping failures in metadata, and applies the configured overlay-target limit.

The harness layer adds a second implementation boundary. The prompt exposes a `HarnessDecision` contract for candidate adjudication, while the public runtime output remains mapped through the stable `HelpResponse` path. `plan_harness_action` then checks the chosen step, candidate evidence, allowed overlay targets, evidence references, and final state consistency. It can accept a model proposal, repair it to a safer target, force text-only guidance, or reject overlay output. This is the concrete implementation of the architecture principle that the LLM is not the final authority.

Bounded overlay execution. The final executor accepts only actions of type `overlay`. It rejects control injection, key emulation, empty targets, unmappable targets, targets outside the UI allowlist, and evidence-required overlays without evidence references. The current runtime supports configurable multi-target guidance. Targets beyond the configured limit are dropped and recorded rather than silently executed. When the DCS sender emits acknowledgements, requested, applied, and failed overlay events are normalised into the same JSONL event stream as the rest of the help cycle.

5.3 Deterministic Fallback

Fallback is implemented as a conservative degradation path. It is used when the model is unavailable, JSON extraction fails, schema validation fails, response mapping rejects the proposed action, or the harness cannot produce a grounded overlay plan. The fallback examines the active procedure step, unsatisfied gates, recent actions, UI target bindings, and evidence availability.

The important distinction is between hard and soft blockers. A hard blocker means that observed evidence shows a required condition is false. A soft blocker means that the required source is missing or not observable. The fallback can give corrective guidance for hard blockers, but it avoids claiming that a learner has failed a step when the necessary evidence is absent. Overlay output is suppressed unless the local procedure rules identify

a unique valid target, or a bounded multi-target plan is explicitly allowed by the current configuration.

This fallback path also preserves the thesis claim boundary. A help cycle may continue without a model or without vision evidence, but the resulting guidance is less ambitious: it relies on observed telemetry, procedure gates, and deterministic step hints rather than on invented visual or pedagogical interpretation.

5.4 VLM Visual Fact Extraction Runtime

The VLM runtime is implemented as a visual fact extractor, not as a tutor. The current procedure pack marks S08, S15, S18, and S19 as vision-priority steps. For those steps, the live runtime may request composite-panel capture before assembling the final help context. For other steps, stale visual facts are not allowed to override fresher telemetry or procedure-gate evidence.

A capture request is sent to the DCS-side vision sidecar, which renders the left DDI, AMPCD, and right DDI into a composite image. The Python-side buffered vision session keeps recent observations and selects frames around the help trigger. In live mode the default synchronization window is 250 ms; replay uses a narrower deterministic window. The selected frame metadata records frame identifiers, capture time, synchronization status, delta from the trigger, and stale-frame flags.

The selected image frames are passed to the visual fact extractor. The extractor builds a prompt from the current 13-fact ontology, the composite layout, and fact-specific decision boundaries. The model response is parsed as JSON, sanitized, and validated against a schema whose fact identifiers come from the visual-facts pack. Unknown fact IDs, invalid states, incomplete fact objects, and non-schema fields are ignored or reported in metadata. Accepted facts are restricted to `seen`, `not_seen`, and `uncertain`.

Validated visual facts are merged into the runtime vision snapshot using the ontology's sticky and expiry policy. This implementation detail matters because some cockpit display states are transient: a fact may need to remain available briefly for a procedure gate or harness decision, while stale facts must expire before they can mislead later steps. If multimodal extraction is disabled or the VLM call fails, the help cycle continues with telemetry, retrieval, and fallback evidence only.

5.5 Replay and Logging Infrastructure

The implementation treats logging as part of the evidence chain. Runtime events are written to a JSONL store with immediate flush after each append. A help cycle therefore leaves an auditable trail of trigger metadata, telemetry context, vision selection, prompt construction metadata, model and mapping status, harness decisions, overlay execution, and failure reasons.

Replay uses the same boundary discipline as live operation. Pre-recorded DCS-BIOS captures can be replayed through the runtime, optionally with vision frame directories, while the procedure pack, UI map, telemetry map, source policy, and harness specs remain explicit inputs. The `fa18c_startup_v04` replay suite combines five scenario cases with live-fixture regression entries that cover stale VLM evidence, wrong-target prevention, moving-control settling, completion-already-true cases, and interaction-text semantics.

These logs and replay suites do not demonstrate learning outcomes. Their role is technical: they make it possible to reproduce a help-cycle decision, inspect why an overlay was accepted, repaired, dropped, or rejected, and support the RQ1 readiness evidence reported in Chapter 6.

5.6 Live Runtime Orchestration

The live runtime connects the implementation pieces through a shared help-cycle path. A help trigger first creates a telemetry snapshot from DCS-BIOS frames. The telemetry pipeline enriches raw observations into runtime variables and recent deltas. If the current step requires visual confirmation, a capture request is issued and the buffered vision session selects synchronized frames. The procedure engine and gate evaluator produce deterministic candidates and missing-condition diagnostics. These observations are assembled into an evidence packet, retrieval adds source-controlled procedural snippets, and the prompt builder constructs the model request.

The text model returns a structured response rather than a free-form command. That response is parsed, schema-validated, mapped to a runtime tutor response, checked by the harness, and converted into at most the configured number of overlay actions. The overlay executor dispatches only validated overlay intents to DCS and records acknowledgements or failures. The complete help cycle, including fallback and repair metadata, is appended to the JSONL log.

This orchestration is also used by replay and regression tooling where possible. The shared path is important for the thesis because the evaluation is not based on a separate mock implementation of the tutor. The same contracts that constrain live tutoring also produce the logs, replay outputs, and quality gates used to evaluate technical readiness.

6 Evaluation

This chapter evaluates the thesis through three evidence lines, each with a different claim strength. The VLM benchmark line answers RQ2 by measuring structured visual fact extraction on independent holdout screenshots. The replay and harness line answers RQ1 by checking whether the evidence-grounded help cycle can be reproduced, validated, and bounded by regression fixtures. The VR pilot line answers RQ3 through descriptive observations from nine participant trials. These lines are deliberately not interchangeable: component accuracy is not a learning outcome, regression coverage is not a human-subject result, and the pilot is formative rather than inferential.

Figure 6.1 gives the chapter-level evidence map. It shows how benchmark artifacts, replay/harness logs, and pilot exports enter the evaluation without collapsing them into a single kind of proof.

6.1 RQ2: VLM Technical Evaluation

The technical evaluation addresses whether small-data LoRA adaptation can make a VLM reliable enough for the narrow task required by the tutor: extracting structured visual facts from F/A-18C cockpit display screenshots. The model is not evaluated as a tutor and is not credited with procedural reasoning. It is evaluated as a perception component whose output becomes one evidence source in the help cycle. All metrics are computed by the automated benchmark harness described in Section 4.6.

6.1.1 Dataset and Training Summary

The visual fact extraction task evolved across two generations of fact ontology. The first generation (v1) defined 8 visual facts and established the initial training pipeline. The second generation (v2) expanded the ontology to 13 facts, aligned with the runtime contract. Table 6.1 summarises the six annotated datasets.

The v2 training recipe combines Run-003 (220 reviewed tasks) with Run-005 (122 reviewed tasks, collected to rebalance under-represented fact states), with Run-005 ap-

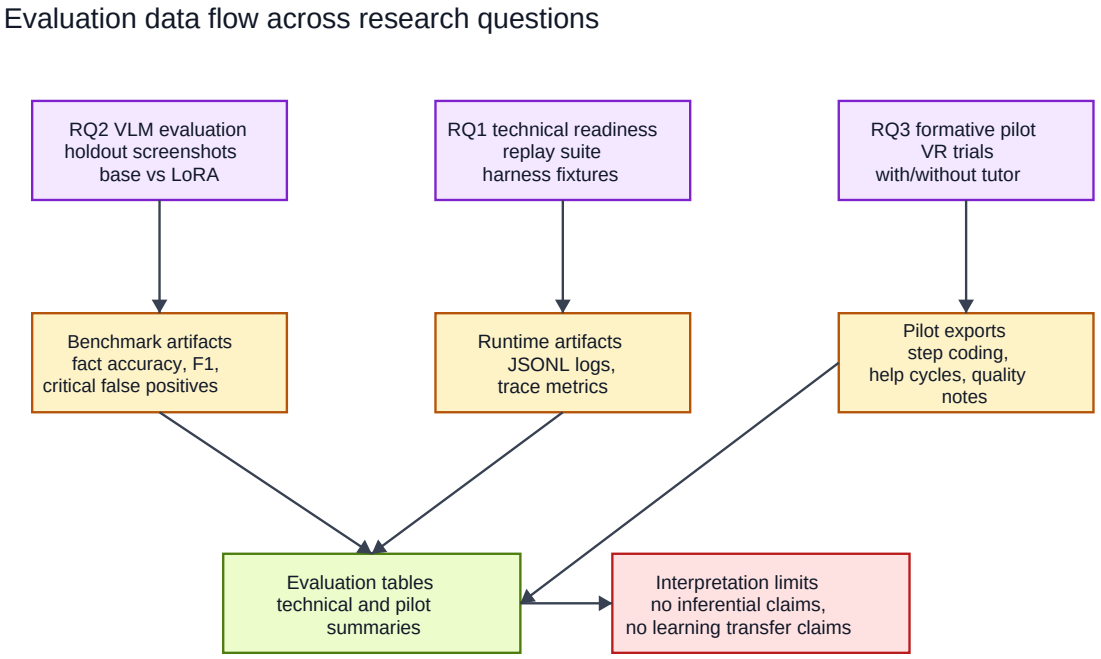

Figure 6.1: Evaluation and data flow across research questions. VLM benchmark artifacts support RQ2, replay and harness logs support RQ1 technical readiness, and VR pilot exports support the descriptive RQ3 findings.

Table 6.1: Summary of annotated visual fact datasets.

Source	Run	Facts	Tasks	Bilingual	Role
v1 train	Run-001	8	180	yes	Initial training
v1 holdout	Run-002	8	50	no	First holdout
v2 newfacts holdout	Run-002	13	50	no	Runtime-aligned holdout
v2 train	Run-003	13	220	yes	Main training source
v2 rebalance	Run-005	13	122	yes	Composition rebalance
v2 random holdout	Run-004	13	100	no	Independent holdout

plied twice (Run-005×2). Bilingual export (EN + ZH) yields a combined training set of **928 rows**. The two v2 holdouts, Run-002 newfacts (50 samples) and Run-004 random (100 samples), are English-only and have verified **zero exact overlap** with the training union. The benchmark therefore tests adaptation beyond exact image reuse, while remaining within the same simulator cockpit and procedure domain.

6.1.2 Qwen3.5 13-Fact V2 Results

The primary technical result is the Qwen3.5-9B LoRA adapter trained on Run-003 + Run-005×2 (928 rows, 4 epochs, learning rate 2×10^{-4}, LoRA $r = 16$, $\alpha = 16$). This is the strongest RQ2 evidence because it compares the adapted model against the corresponding base model on two independent holdouts with the runtime-aligned 13-fact ontology. Table 6.2 reports the result.

Table 6.2: 13-fact v2 benchmark: Qwen3.5-9B Base vs. LoRA on two holdout sets.

	Run-002 newfacts (50)			**Run-004 random (100)**		
Metric	**Base**	**LoRA**	**Δ**	**Base**	**LoRA**	**Δ**
JSON valid	1.0	1.0	—	0.96	1.0	+0.04
Schema valid	1.0	1.0	—	0.96	1.0	+0.04
Fact accuracy	0.8677	**0.9908**	+0.1231	0.8038	**0.9931**	+0.1892
Macro F_1	0.4513	**0.6515**	+0.2002	0.4393	**0.6100**	+0.1707
Seen F_1	0.5022	**0.9648**	+0.4626	0.5659	**0.9159**	+0.3500
Exact match	0.10	**0.88**	+0.78	0.03	**0.92**	+0.89
Critical FP	11	**4**	−7	70	**8**	−62

The adapted model reaches fact accuracy of 0.9908 on Run-002 and 0.9931 on Run-004, with sample exact match rates of 0.88 and 0.92. More important for the tutor is the change in critical false positives, because a false `seen` state can make a later help-cycle decision appear grounded when the display evidence is not actually present. Critical false positives drop from 11 to 4 on Run-002 and from 70 to 8 on Run-004. The base model's larger Run-004 failure is not a formatting problem alone: it includes four schema-invalid responses and a large number of rare-state hallucinations. The LoRA adapter restores JSON/schema reliability and reduces this hallucination pattern to a smaller set of interpretable residual errors.

The remaining LoRA critical false positives are concentrated on completion signals rather than page-presence signals. In practice, this means the adapted VLM is strong enough to support the visual-fact pipeline, but not strong enough to be treated as an

independent source of procedural completion. That boundary matches the architecture in Chapter 3: visual facts are evidence inputs, not procedure decisions.

6.1.3 Gemma-4 and Qwen3.6 Backbone Comparison

The same Run-003 + Run-005×2 recipe was also applied to Qwen3.6-27B and Gemma-4-31B. This comparison is supplementary: it checks whether the data and ontology recipe transfers across backbones, while the Qwen3.5 base-vs-LoRA line remains the primary technical result. Table 6.3 summarises the LoRA-only comparison.

Table 6.3: 13-fact v2 results across three backbones (LoRA models only).

	Run-002 newfacts (50)			**Run-004 random (100)**		
Metric	**Q3.5-9B**	**Q3.6-27B**	**G-4-31B**	**Q3.5-9B**	**Q3.6-27B**	**G-4-31B**
Fact accuracy	**0.9908**	0.9877	0.9492	**0.9931**	0.9892	0.9715
Seen F_1	**0.9648**	0.9479	0.8123	**0.9159**	0.9147	0.7959
Exact match	**0.88**	0.86	0.44	**0.92**	0.86	0.74
Critical FP	4	3	**0**	**8**	8	13

Qwen3.6-27B produces a close result but does not surpass the smaller Qwen3.5 adapter on the main metrics. This suggests that, for this narrow visual-fact task, a larger backbone does not automatically compensate for the value of a well-matched small-data adaptation recipe.

Gemma-4-31B shows a different error style. On Run-002 it produces zero critical false positives, but its seen-state recall and sample exact match are lower than Qwen3.5. On Run-004, the Gemma adapter improves substantially over its base line but still records more critical false positives than the Qwen3.5 adapter. The comparison therefore supports a conservative conclusion: the 13-fact data recipe transfers across backbones, but the deployment choice remains backbone-specific. Qwen3.5 is the best-performing configuration in these local benchmarks; Gemma is more conservative in one holdout but less complete.

6.1.4 Error Analysis

The Qwen LoRA error profile also explains why the ontology revision in Chapter 4 matters. Coarse page-presence facts are nearly solved. Fine-grained state indicators are mostly reliable after adaptation, but completion-type signals remain the main residual risk. Small text facts are usable but still sensitive to ambiguous display states.

The base model's dominant failure mode is unidirectional hallucination on rare states. In the larger Run-004 holdout, the FCS-MC intermediate result appears in only 7 of 100 samples, yet the base model asserts this state as `seen` in 60 additional samples. The LoRA adapter removes this diffuse hallucination pattern almost entirely. The remaining errors are fewer and more diagnosable, which is precisely the kind of error profile needed for evidence-grounded tutoring: the runtime can reason about a bounded set of known visual-fact risks instead of treating raw VLM text as opaque authority.

6.2 RQ1: System-Level Technical Readiness

RQ1 concerns whether the runtime can integrate evidence sources in a technically reliable and auditable way. The relevant evidence is therefore not participant performance, but reproducibility of help-cycle decisions, regression coverage of known failure modes, and export quality for later analysis.

6.2.1 Replay Evaluation

The replay evaluation suite defines five test cases: no-operation, battery-on, battery-and-generator-off, FCS reset and BIT, and INS alignment (see Appendix 9, Table 9.4). Each case includes DCS-BIOS raw captures and, where applicable, per-frame sidecar files for vision-dependent steps. The suite exercises both telemetry-only and vision-assisted replay conditions. Its role is to show that recorded state can be driven through the same procedure pack, telemetry map, UI map, evidence policy, and harness contracts that constrain live tutoring.

6.2.2 Harness Fixture Regression

The step harness (Section 3.6) is covered by 20 live-fixture regression entries in the coverage matrix. Each entry replays a specific live telemetry snapshot through the help cycle and asserts an expected outcome. The fixtures cover five classes of readiness risk: advancement when a step is already complete, wrong-target prevention and repair, prevention of stale VLM evidence leaking into non-visual steps, waiting behaviour for moving/settling controls, and preservation of interaction semantics such as mouse-wheel controls. This coverage does not prove that the tutor is pedagogically effective; it shows that known evidence-grounding failures are captured as reproducible tests.

6.2.3 Experiment Export Infrastructure

The experiment export pipeline (Section 4.7) has been exercised on nine live VR trials, producing trial summaries, step coding, help cycles, action timelines, raw-event copies, and quality-gate reports for each trial. All nine exports have no blocking quality-gate errors, and the recorded procedure-pack, taxonomy, UI-map, and DCS-BIOS-to-UI hashes are consistent across the study. This supports the instrumentation part of RQ1: the runtime can produce audit-ready artifacts from real participant sessions. It also exposes a remaining limitation. One trial, P04, passed in non-strict mode with metadata warnings for missing questionnaire, recording, mission, and prompt linkage fields. Those warnings do not invalidate the task metrics used below, but they show that the participant/session metadata schema should be tightened before a larger controlled study.

6.3 RQ3: Formative VR Pilot Study

A small formative pilot study was conducted with nine participants. Four used the tutor during the F/A-18C cold-start task and five practised without tutor assistance. The purpose of this pilot is descriptive: it shows how the runtime, overlay guidance, visual evidence path, and export pipeline behaved in live VR use. It is not a powered controlled experiment, and it does not support inferential claims about learning gains, workload, retention, transfer, or task speed. Methodology details are reported in Section 4.7.

6.3.1 Participant and Trial Overview

Table 6.4 summarises each participant's condition, completion status, manually reviewed step completion, observed duration, and key interaction counts. The table uses manual step coding as the authoritative source where automatic reconstruction and human review disagree.

Completion. All four with-tutor participants reached the final 33-step state under manual review. None of the five without-tutor participants completed the full procedure; they stopped after completing 19–29 steps. The most commonly missed without-tutor steps were S18 (FCS-MC BIT page), S19 (FCS BIT final GO), and S27 (flap switch to AUTO), each missed by all five without-tutor participants.

Observed duration. With-tutor trials ranged from 317 s to 1033 s (median 387 s). Without-tutor trials ranged from 218 s to 2015 s. These values are reported as trace durations only. Because the without-tutor trials ended at different procedure points, the

Table 6.4: Participant and trial overview. Observed duration is the wall-clock interval from first telemetry event to last; it is not a completion time measure for incomplete trials.

P	Condition	Completed	Steps	Dur. (s)	Help	VLM	Ovrl.	Fallbk.
P01	without_tutor	no	23/33	2015	0	0	0	0
P02	without_tutor	no	27/33	739	0	0	0	0
P03	without_tutor	no	24/33	243	0	0	0	0
P06	without_tutor	no	29/33	269	0	0	0	0
P07	without_tutor	no	19/33	218	0	0	0	0
P04	with_tutor	yes	33/33	442	19	6	22	6
P05	with_tutor	yes	33/33	317	7	2	6	2
P08	with_tutor	yes*	33/33	332	7	5	10	4
P09	with_tutor	yes	33/33	1033	44	10	44	9

*P08: auto-summary reports incomplete; manual coding confirms 33/33 completed (see data-quality note below).

durations are not completion-time comparisons between conditions.

6.3.2 Procedural Errors

Table 6.5 reports manual error counts per participant.

Table 6.5: Manual procedural error counts per participant (from manual step coding). OM = omission, CO = commission, OR = order error, PA = prompting/ assistance error, SV = state violation.

P	Condition	OM	CO	OR	PA	SV
P01	without_tutor	8	0	1	0	1
P02	without_tutor	0	0	0	1	0
P03	without_tutor	7	0	2	0	2
P06	without_tutor	2	0	0	2	0
P07	without_tutor	11	0	0	0	0
P04	with_tutor	0	3	0	3	1
P05	with_tutor	0	3	0	3	0
P08	with_tutor	0	0	0	0	0
P09	with_tutor	0	3	0	2	0

Omission errors. The without-tutor condition shows a clear omission pattern: four of five participants recorded omission errors, and three participants recorded seven or more. In contrast, the with-tutor condition recorded zero manual omission errors; every required step was ultimately completed under manual review.

Commission and prompting errors. The with-tutor condition shows a different error profile. CO and PA errors appear in P04, P05, and P09, while P08 has no manually coded procedural errors after review. These errors are assistance-coupled: the participant often followed guidance, but the guidance or its timing did not fully match the procedure specification or current state. This is an important formative result. The tutor condition was associated with continued progression through the procedure, but it also introduced new categories of guidance risk that should be reduced before a larger study.

Error totals. Without-tutor participants averaged 7.4 manual errors; with-tutor participants averaged 4.5. This difference is descriptive only and should not be read as an inferential treatment effect. The more important finding is qualitative: without-tutor errors are dominated by omissions, while with-tutor errors shift toward commission, prompting/assistance, and one state violation.

6.3.3 Tutor Interaction

Table 6.6 summarises tutor interaction metrics for the four with-tutor trials.

Table 6.6: Tutor interaction summary (with-tutor condition only).

P	Help cycles	VLM calls	Overlay exec.	Overlay rej.	Fallback
P04	19	6	22	0	6
P05	7	2	6	0	2
P08	7	5	10	0	4
P09	44	10	44	0	9
Total	77	23	82	0	21

All 77 help cycles used the model generation path; no cycle fell back to a deterministic-only response. VLM calls totalled 23 across the four with-tutor trials. The current procedure pack treats S08, S15, S18, and S19 as vision-priority steps, but the pilot logs also reveal occasional unnecessary VLM calls around non-visual or already-progressed states. That over-triggering is a runtime tuning issue rather than a participant outcome.

Overlay rejection rate was zero: all 82 executed overlay actions passed the runtime's evidence-grounding, allowlist, and schema checks. The 21 fallback events ($21/77 = 27\%$ of help cycles) were not deterministic-only fallbacks. They were harness or validator repairs of the model's proposed overlay plan, such as remapping a coarse display target to the more specific BIT-root sub-state target. This supports the RQ1

claim that the live system can repair model output before it reaches the overlay, while also showing where the model still needed runtime correction.

6.3.4 Data-Quality Notes

1. **P08 auto-summary correction.** The automated pipeline reported P08 as incomplete (5/33 steps). Manual telemetry review confirmed all 33 steps reached required states. The auto-pipeline failed to reconstruct completion evidence for vision-priority steps from passive telemetry. Manual coding overrides the auto-summary; P08 is reported as completed with 0 manual errors.

2. **P02 questionnaire exclusion.** P02's questionnaire JSON file contains an internal participant-id mismatch, indicating mislabelling. P02 is excluded from any workload, TLX, or confidence summary. Task metrics from the trial summary and manual step coding are retained.

3. **Quality-gate interpretation.** All nine participant exports passed without blocking quality-gate errors. P04 nevertheless has metadata warnings for missing questionnaire, recording, mission, and prompt linkage fields. These warnings limit questionnaire-level analysis and traceability, but they do not contradict the task metrics reported in Tables 6.4 and 6.5.

4. **Duration interpretation.** Without-tutor trials ended at different procedure points when participants could not progress further. Observed durations are reported for completeness but do not support completion-time comparison between conditions.

5. **Non-random assignment.** Participants were assigned to conditions based on scheduling availability, not randomisation. Prior DCS and VR experience varied across participants (see Appendix 9, Table 9.5).

6.3.5 Summary of Pilot Findings

The pilot provides three pieces of formative evidence. First, the tutor condition supported procedural progression in these four trials: all four with-tutor participants completed the reviewed 33-step procedure, while none of the five without-tutor participants did. Second, the error pattern changed. The tutor condition removed coded omissions, but

it did not remove all procedural problems; assistance-coupled CO and PA errors appeared in three of four with-tutor trials. Third, the live tutoring infrastructure operated through 77 help cycles and 82 overlay executions without overlay rejection, while logging the repairs and fallback decisions needed for audit.

Across the full chapter, the strongest empirical claim is the RQ2 VLM result: small-data LoRA adaptation produced a much more reliable 13-fact visual extractor on local holdouts. RQ1 is supported by replay, harness, and export evidence showing that the runtime can be audited and regression-tested. RQ3 is supported by a completed formative VR pilot, but only at descriptive strength. The results do not establish learning improvement, workload reduction, faster skill acquisition, retention, or transfer. A larger, randomised, controlled study would be required for those claims.

7 Discussion

Chapter 6 reported three evidence lines with different claim strengths: visual-fact benchmarks for RQ2, replay and harness readiness evidence for RQ1, and a formative VR pilot for RQ3. This chapter interprets those evidence lines together. The main claim is not that the tutor has already been proven to improve learning. Rather, the thesis shows how a procedural tutor can be made technically auditable by placing telemetry, procedural rules, retrieved sources, VLM-derived visual facts, and harness validation between the learner and the foundation-model response.

Section 7.1 explains how the implemented work differs from the original proposal. Section 7.2 interprets the technical evidence. Section 7.3 discusses the pilot findings within their formative boundary. Section 7.4 draws the main design lesson from the ontology evolution. Sections 7.5 and 7.6 state the limits of the thesis and the most direct next research steps.

7.1 Evolution from Proposal to Implemented System

The original thesis proposal [24] described a VR-first tutoring system for the Meta Quest 3, with gaze, hand tracking, voice input, a RAG+LLM backend, and a controlled three-condition human-subject study. The final thesis keeps the immersive procedural tutoring motivation, but the center of the work moved. The implemented contribution is best understood as an evidence-grounded tutoring runtime and visual-fact extraction method, with VR used for a formative pilot rather than for a controlled effectiveness claim.

This shift was not only an engineering convenience. The cold-start procedure made a prior problem visible: before a tutor can give useful procedural help, it must know which state it is allowed to trust. Some required evidence is exported as telemetry through DCS-BIOS. Some evidence is visible only on cockpit displays. Some steps are manual, out of layout, or unsuitable for automatic confirmation. The thesis therefore had to build the evidence layer first: procedure packs, gate inference, evidence packets, retrieval

boundaries, VLM visual facts, schema-constrained model output, harness validation, overlay allowlists, and replayable logs.

The VLM work grew out of the same pressure. The proposal assumed that simulator state would be sufficient for procedural tracking. In practice, display-page content such as alignment and BIT states had to be observed visually. This led to the 13-fact ontology, the reviewed screenshot datasets, the LoRA adaptation pipeline, and the independent holdout benchmarks. These became a core contribution because they define how a foundation model can be used as a bounded perception component rather than as an unconstrained judge of procedure state.

The completed VR pilot partially returns to the original proposal's human-subject ambition. It shows that the evidence-grounded runtime can be exercised in headset with real participants and that the export pipeline can capture the data needed for later analysis. It does not, however, complete the controlled three-condition study proposed at the beginning of the project. With $n = 9$, non-random assignment, and unresolved questionnaire data for one participant, the pilot is formative and descriptive.

7.2 Interpretation of Technical Evidence

The strongest technical result is the RQ2 visual-fact benchmark. The primary Qwen3.5-9B LoRA line reaches approximately 0.99 fact accuracy on both independent 13-fact holdouts, while reducing critical false positives sharply relative to the base model. Supplementary Qwen3.6-27B and Gemma-4-31B results show that the same data-and-ontology recipe is not tied to a single backbone, although the thesis does not claim broad model-family generality. The important interpretation is narrower and more useful: a small, reviewed, ontology-aligned dataset can make a VLM substantially more reliable for cockpit visual-fact extraction than the corresponding base model on local holdout data.

That result matters because the tutoring runtime treats VLM output as evidence, not as authority. A visual fact such as a visible page, label, or status text is one input to downstream step inference. The procedure engine still combines visual facts with telemetry, procedure order, temporal context, and pack-defined gates. This separation is what prevents the benchmark from becoming an unsupported pedagogical claim. The VLM result supports the perception layer of the tutoring system; it does not by itself prove that the tutor teaches better.

The RQ1 replay and harness evidence supports a different part of the thesis. The replay cases, live-fixture coverage, pack-derived schemas, response mapping, and overlay

allowlists show that the implemented runtime can constrain model output before it reaches the learner. This is a technical readiness and auditability claim. It means that help cycles can be replayed, inspected, and checked against explicit evidence contracts. It does not mean that every future model response will be pedagogically optimal, nor that the runtime has been validated across simulators or procedures.

Taken together, the technical evidence supports the architecture's central premise: foundation models can be useful in procedural tutoring when their role is bounded by explicit evidence production and validation. The thesis does not ask the LLM to be the source of truth. It asks the LLM to generate a response inside a runtime where the observable state, procedural pack, retrieved sources, and harness define what the response may safely claim or highlight.

7.2.1 What the Technical Evidence Does Not Prove

The technical evidence does not show learning gains, retention, transfer, reduced workload, or superior training outcomes. The VLM benchmarks measure single-frame visual-fact extraction. The replay and harness tests measure whether evidence contracts and output constraints behave as expected. Both are necessary for a reliable tutor, but neither is sufficient for a human learning claim. A learner could receive accurate, grounded help and still fail to form a durable procedural understanding. Conversely, a learner might learn from a less instrumented system if it happens to fit their prior knowledge and strategy.

The evidence also does not yet isolate the value of each component. This thesis does not contain an ablation comparing telemetry-only tutoring, vision-augmented tutoring, different retrieval policies, or different harness strictness levels. The evaluation therefore supports an integrated artifact claim: the implemented system shows a technically auditable way to combine these elements. It does not rank every design choice against alternatives.

7.3 Interpretation of Pilot Findings

The formative VR pilot provides descriptive evidence about live use of the system. In the reviewed data, all four with-tutor participants completed the 33-step procedure, while none of the five without-tutor participants completed it. Manual coding also shows zero omission errors in the tutor condition and a mean of 5.6 omission errors in the without-

tutor condition. These observations are consistent with the intended function of the tutor: it keeps the current procedure step visible, retrieves relevant procedural context, and can point the learner toward cockpit controls.

The same data also show that assistance does not eliminate procedural error. Commission and prompt-assistance errors remained in the tutor condition. This is an important finding because it points to a more precise design problem than "does the tutor help?" The remaining problem is how flexible model-generated guidance, deterministic harness repair, visual-state uncertainty, and human action timing align with a strict procedure specification. The tutor can reduce missed-step patterns while still leaving room for wrong timing, incomplete execution, or over-trust in assistance.

The live help-cycle logs support the infrastructure claim. Across the four with-tutor trials, the system recorded 77 help cycles, 23 VLM calls, 82 overlay executions, zero overlay rejections, and 21 fallback or repair events. The zero overlay rejection count should not be read as "nothing went wrong." Rather, it means that proposed overlay actions stayed within the configured allowlist after schema validation and harness processing. The repair events are evidence that the constraint layer was active, not evidence that model output was always directly correct.

The pilot therefore has two roles in the thesis. First, it shows that the instrumented tutoring runtime can be used in a live VR setting and produce analysable logs. Second, it reveals the kind of human-system interaction data that a later controlled study should measure more carefully. It does not support causal claims about learning, workload, retention, transfer, or task speed.

7.4 Ontology Design Lessons

The ontology evolution is the clearest design lesson in the thesis. The first visual-fact ontology mixed observable facts with procedural conclusions. For example, `ins_go` asked the VLM to decide whether an alignment state had effectively been reached. That target was not simply visual. It depended on particular display text, the absence of other display states, and the procedure phase. The failure pattern in the first benchmark made the boundary problem visible: a single-frame VLM was being asked to perform both perception and procedure reasoning.

The 13-fact ontology corrected that boundary. It decomposed compound targets into lower-level visual observations, such as whether the ground-alignment text or OK text is visible, and decomposed FCS-MC states into visually distinguishable stages. The

downstream runtime then interprets those observations together with telemetry and procedural context. The design principle is therefore:

> **Visual fact targets should be low-level, locally verifiable visual propositions. Higher-level procedural state should be inferred downstream from visual facts, telemetry, temporal context, and procedure rules.**

This principle matters beyond label naming. It affects annotation quality, training signal quality, error diagnosis, and runtime safety. A human reviewer can judge whether a piece of text is visible in a screenshot without knowing the procedure history. The same reviewer cannot always judge whether a procedure state is complete without knowing what happened before the image. When the fact contract respects that distinction, the model's output becomes easier to review and safer to combine with deterministic gates.

Residual false positives on completion-type facts show the remaining edge of the method. Some states are visually similar across intermediate and final phases, especially when numeric readouts or transient display states are involved. Those cases may require more balanced data, higher-resolution crops, repeated observations, or explicitly temporal VLM inputs. The thesis therefore does not claim that the 13-fact ontology solves visual grounding generally. It offers a tested design discipline for one demanding procedure and a concrete method for discovering when a visual target is still too close to a procedural conclusion.

As a heuristic for future domains, the rule is simple: if a fact cannot be labelled from one image without knowing the procedure step, it is probably too high-level for the VLM contract. It should be decomposed into visible atoms or deferred to the procedure engine. This is the point at which the thesis moves from a case-specific implementation detail to a reusable representation lesson.

7.5 Limitations

The contribution should be read inside the following limits.

7.5.1 Evaluation Scope

All datasets, replay cases, and pilot trials concern one procedure: the F/A-18C cold-start. The visual extractor is trained and evaluated on a fixed composite cockpit panel

with the same aircraft type and rendering environment. The holdout sets are independent sessions with zero training overlap, but they still measure within-domain generalisation. They do not establish cross-procedure, cross-aircraft, or cross-simulator generality.

The largest training recipe contains 928 bilingual SFT rows after the Run-005 oversampling step. This is deliberately small and practically useful for a domain-specific adaptation study, but it does not identify the minimum data requirement, the performance ceiling, or the effect of each recipe component. The thesis does not include ablations for bilingual rows, oversampling, LoRA rank, composition rebalance, or alternative fine-tuning methods.

7.5.2 System and VLM Scope

The VLM is evaluated as a single-frame visual-fact extractor. It does not decide procedure progress, select final tutor actions, or control the overlay directly. Residual critical false positives on completion-like facts remain important because a false visual observation can still influence downstream reasoning if it is not checked by telemetry, temporal confirmation, or harness constraints. Sticky facts and confirmation policies mitigate this risk, but their complete end-to-end effect has not been isolated experimentally.

The backbone comparison covers Qwen3.5-9B, Qwen3.6-27B, and Gemma-4-31B under a matched LoRA-style adaptation recipe. This is useful model-diversity evidence within the scope of the project, but it does not cover other VLM families, larger models, full fine-tuning, alternative adapters, or non-cockpit visual domains.

The runtime is implemented and exercised through replay, fixtures, desktop DCS, and the Quest 3 pilot. However, headset latency, overlay legibility, gaze or hand-tracking accuracy, and expert-rated response usefulness were not measured as controlled system-level outcomes.

7.5.3 Pilot Scope

The pilot contains nine participants: four with tutor and five without tutor. Assignment was not randomised, prior DCS experience was not balanced, and the sample is too small for inferential statistics. Without-tutor trials stopped at different procedure points, so their durations are not comparable to completed with-tutor trials.

Questionnaire and workload evidence is incomplete. P02's questionnaire metadata could not be confidently attributed, and NASA-TLX integration was not part of the

completed export pipeline. The pilot therefore supports task-log and coding observations, not workload, confidence, or subjective-usability conclusions.

Finally, manual review inferred without-tutor completion from passive telemetry gates and the action timeline. Because the active VLM pathway was not available in that condition, visually evidenced steps are less certain. This limitation does not invalidate the formative finding, but it shows why the next study needs stricter instrumentation and coding.

7.5.4 Scope Summary

Within these limits, the thesis supports four claims: an evidence-constrained procedural tutoring runtime can be implemented with explicit architecture boundaries; a small-data VLM adaptation pipeline can produce reliable visual-fact extraction on local holdouts; the visual ontology evolution provides a concrete design lesson about perception and reasoning; and the VR pilot shows that the system can be exercised with participants while producing analysable logs. It does not establish controlled educational effectiveness.

7.6 Future Work

The limitations above lead to a focused research agenda.

7.6.1 Controlled Human-Subject Evaluation

The next study should compare the grounded tutor against a baseline condition with randomised assignment, balanced participant metadata, pre-defined coding rules, and complete questionnaire and workload capture. A two-condition design would be more realistic than the original three-condition proposal for the next iteration, because the current priority is to establish whether the completed runtime produces measurable differences in completion, errors, workload, and retention under controlled conditions.

7.6.2 Ontology Extension to Other Procedures

The 13-fact ontology should be extended to additional procedures only after a new task analysis identifies which steps are telemetry-visible, vision-priority, or manual/out-of-layout. Each new procedure needs its own visual fact contract, reviewed data, holdout

benchmark, and runtime bindings. This is the appropriate test of whether the ontology discipline transfers beyond cold-start.

7.6.3 Temporal and Multi-Frame Visual Reasoning

Several residual errors involve states that are difficult to disambiguate from a single screenshot. Multi-frame inputs, short visual histories, or repeated confirmation gates could help distinguish intermediate and final display states. This would require changes to capture, annotation, model input format, and benchmark metrics.

7.6.4 System-Level Help-Cycle Evaluation

A separate evaluation should measure the full help cycle rather than individual components: trigger-to-overlay latency, correctness of highlighted targets, frequency and type of harness repair, response usefulness under expert review, and whether vision-priority steps improve over telemetry-only baselines. This would connect the RQ1 and RQ2 technical evidence more directly to RQ3.

7.6.5 In-Situ Correction and Adaptation

The current VLM adaptation pipeline is offline. Future versions could collect instructor or learner corrections when a visual fact is wrong and feed those corrections into later review or incremental adaptation. This direction requires careful safeguards because in-situ labels may be noisy, and adaptation must not erase previously reliable behaviour.

7.6.6 Summary of Future Directions

Table 7.1 summarises the main future-work directions and the limitation each direction addresses.

These directions keep the thesis spine intact. The next stage is not to make the foundation model less constrained, but to strengthen the evidence loop around it: broader procedure packs, better visual facts, richer temporal evidence, controlled human evaluation, and logs that make every tutor response auditable after the fact.

Table 7.1: Summary of future work directions.

Direction	**Current status**	**Main challenge**
Controlled human-subject study	Formative pilot completed; causal evidence pending	Randomisation, metadata, workload, and statistical power
Ontology extension	Cold-start ontology evaluated only within current cockpit layout	New task analysis, annotation, and holdout design
Temporal visual reasoning	Current VLM uses single composite-panel frames	Multi-frame data, model capacity, and temporal metrics
System-level help-cycle evaluation	Component and replay evidence available	Defining integrated latency, quality, and repair metrics
In-situ correction and adaptation	Offline review and LoRA pipeline implemented	Noisy labels, forgetting, and safe update policies

8 Conclusion

This thesis began from a practical problem shared by many procedural training settings: a learner often needs help while acting inside a complex simulated system, but useful help must be grounded in what is currently true. A fluent foundation-model response is not enough when the next correct action depends on telemetry, procedure order, display state, and earlier actions. The central answer developed in this thesis is that foundation models can be made suitable for procedural tutoring when they are placed inside an auditable evidence loop.

The F/A-18C cold-start task provided a demanding case for that idea. It required the tutor to combine telemetry-visible cockpit state, visual evidence from display pages, retrieved procedural knowledge, manual or out-of-layout boundaries, and validation before producing overlay or textual guidance. The result is not a claim that learning effectiveness has already been proven. It is a technical and methodological contribution: a way to make model-assisted procedural help state-aware, inspectable, and bounded by available evidence.

8.1 Summary of Completed Work

The completed work answers the three research questions as follows.

RQ1: evidence integration and tutoring architecture. The thesis shows how simulator telemetry, retrieved procedural sources, and visual fact observations can be integrated into a constrained help cycle. The runtime builds an evidence packet before model generation, infers candidate procedure steps from declarative gates, asks the model to respond inside a pack-derived schema, validates or repairs the response through a harness, and executes only allowlisted overlay actions. Logging and replay then make the resulting help cycle inspectable after the fact. This answers RQ1 at the level of technical auditability: the LLM is not the authority for procedure state, but a generator constrained by evidence, schema, harness, and procedure-pack rules.

RQ2: visual fact extraction and ontology design. The thesis also shows that small-data VLM adaptation can support reliable visual-fact extraction for the cockpit display states required by the tutor. The 13-fact ontology and LoRA training pipeline produce the strongest primary result on the Qwen3.5-9B line, with approximately 0.99 fact accuracy on two independent 13-fact holdouts and sharply reduced critical false positives relative to the base model. Supplementary Qwen3.6-27B and Gemma-4-31B results support the same data-and-ontology recipe within the evaluated backbones. The more general lesson is representational: the VLM should report locally verifiable visual facts, while procedural completion should be inferred downstream from visual facts, telemetry, time, and procedure context.

RQ3: formative VR pilot evidence. The completed Quest 3 pilot provides formative evidence about whether the runtime and instrumentation can be exercised with participants in immersive simulation. In the reviewed data, the four with-tutor participants completed the 33-step procedure, while the five without-tutor participants did not. Omission errors disappeared in the tutor condition, while assistance-coupled errors remained. The help-cycle logs also show that the tutor can operate live with VLM calls, overlay actions, harness repairs, and replayable exports. These findings answer RQ3 descriptively. They support feasibility, instrumentation readiness, and error-pattern analysis; they do not establish causal learning gains, workload reduction, retention, transfer, or speed improvement.

Together, these answers define the thesis contribution. First, the work provides an evidence-grounded tutoring runtime pattern for procedural simulation, where generated help is downstream of explicit state evidence and validation. Second, it contributes a visual-fact ontology and adaptation method that separates VLM perception from procedural reasoning. Third, it delivers a pilot-ready evaluation path for studying grounded tutoring with real VR participants. The important point across all three is not the presence of an LLM or VLM by itself, but the boundary placed around foundation-model output.

The scope remains deliberately bounded. The evaluation covers one procedure, one cockpit layout, local visual holdouts, replay and harness readiness, and a small non-random pilot. The current evidence supports technical reliability and auditability more strongly than educational effectiveness. It does not show that the tutor generalises across domains or that the pilot differences were caused by the tutor alone. Those boundaries are not weaknesses to hide; they are what make the contribution inspectable.

8.2 Future Work

The immediate next step is a controlled human subject evaluation. Such a study should keep the current export and replay infrastructure, but add randomised assignment, balanced participant metadata, complete questionnaire and workload capture, pre-defined coding rules, and a sample large enough for inferential analysis. Only that kind of study can test whether, and how, the technically grounded help cycle changes learning, workload, retention, or transfer.

The technical agenda is equally clear. The procedure-pack and visual-fact approach should be extended to additional procedures, but each extension needs new task analysis, evidence-source mapping, visual ontology design, reviewed data, and independent holdout evaluation. The VLM pipeline should also move beyond isolated screenshots where procedure state depends on temporal evidence: short visual histories, repeated confirmation, or multi-frame inputs are natural next mechanisms for completion-like facts. Finally, the full help cycle should be evaluated as an integrated system, including latency, overlay correctness, harness repair frequency, and expert-rated response usefulness.

A longer-term version of the system could learn from in-situ corrections by instructors or learners. That direction would turn live tutoring into a data collection loop, but it requires careful safeguards: correction labels may be noisy, updates must not erase already reliable behaviour, and every adapted model must still be evaluated against untouched holdouts.

The title of this thesis, *Explain Reality*, captures the final design commitment. A procedural tutor should not merely explain a checklist, and it should not merely generate plausible language about a task. It should explain the learner's current reality: what is observable, which evidence supports it, which procedure rule applies, what remains uncertain, and why the next response is allowed. The contribution of this thesis is an architecture and evidence method for making that explanation auditable.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning*, 2022. doi: 10.48550/arXiv.2204.01691.

[2] Vincent Aleven, Bruce M. McLaren, Jonathan Sewall, and Kenneth R. Koedinger. A new paradigm for intelligent tutoring systems: Example-tracing tutors. *International Journal of Artificial Intelligence in Education*, 19(2):105–154, 2009. doi: 10.3233/IRG-2009-19(2)02.

[3] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2019. doi: 10.1145/3290605.3300233.

[4] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 2023.

[5] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. InstructBLIP: Towards general-purpose vision-language models with instruction tuning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.48550/arXiv.2305.06500.

[6] DCS-BIOS Project. DCS-BIOS documentation, n.d. DCS-BIOS v0.10.0 documentation. Accessed 2026-06-05. Available at: https://dcs-bios.readthedocs.io/en/latest/.

[7] Eagle Dynamics. DCS World, 2013. Digital Combat Simulator. Available at: https://www.digitalcombatsimulator.com/.

[8] Eagle Dynamics. DCS: F/A-18C early access guide, 2023. Updated 18 July 2023. Available at: https://www.digitalcombatsimulator.com/en/downloads/documentation/dcs-hornet_early_access_guide_en/.

[9] David Gunning, Mark Stefik, Jaesik Choi, Timothy Miller, Simone Stumpf, and Guang-Zhong Yang. XAI—explainable artificial intelligence. *Science Robotics*, 4 (37):eaay7120, 2019. doi: 10.1126/scirobotics.aay7120.

[10] Daniel Han and Michael Han. Unsloth: Open-source library for faster LLM fine-tuning, 2024. URL https://github.com/unslothai/unsloth.

[11] Robert T. Hays, John W. Jacobs, Carolyn Prince, and Eduardo Salas. Flight simulator training effectiveness: A meta-analysis. *Military Psychology*, 4(2):63–74, 1992. doi: 10.1207/s15327876mp0402_1.

[12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[13] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, Stephan Krusche, Gitta Kutyniok, Tilman Michaeli, Claudia Nerdel, Jürgen Pfeffer, Oleksandra Poquet, Michael Sailer, Albrecht Schmidt, Tina Seidel, Matthias Stadler, and Gjergji Kasneci. ChatGPT for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103:102274, 2023. doi: 10.1016/j.lindif.2023.102274.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[15] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in Neural Information Processing Systems*, 36, 2023. doi: 10.48550/arXiv.2304.08485.

[16] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods, 2022. URL https://github.com/huggingface/peft.

[17] Antonija Mitrovic. Fifteen years of constraint-based tutors: What we have achieved and where we are going. *User Modeling and User-Adapted Interaction*, 22(1–2):39–72, 2012. doi: 10.1007/s11257-011-9105-9.

[18] Riccardo Palmarini, John Ahmet Erkoyuncu, Rajkumar Roy, and Hosein Torabmostaedi. A systematic review of augmented reality applications in maintenance. *Robotics and Computer-Integrated Manufacturing*, 49:215–228, 2018. doi: 10.1016/j.rcim.2017.06.002.

[19] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.

[20] Maxim Tkachenko, Mikhail Malyuk, Andrey Holmanyuk, and Nikolai Liubimov. Label Studio: Data labeling software, 2020. URL https://github.com/HumanSignal/label-studio.

[21] Kurt VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4):197–221, 2011. doi: 10.1080/00461520.2011.611369.

[22] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. TRL: Transformer reinforcement learning, 2020. URL https://github.com/huggingface/trl.

[23] Bryan Wang, Gang Li, Xin Zhou, Zhourong Chen, Tovi Grossman, and Yang Li. Screen2Words: Automatic mobile UI summarization with multimodal learning. *arXiv preprint arXiv:2108.03353*, 2021. doi: 10.48550/arXiv.2108.03353.

[24] Yu Zhang. Explain reality: Grounded procedural tutoring with foundation models and visual fact extraction in immersive simulation — MSc thesis proposal. TU Clausthal, Institute for Software and Systems Engineering (ISSE). Unpublished manuscript., 2025.

9 Appendix

9.1 Full LoRA Hyperparameter Configuration

Table 9.1 lists the complete hyperparameter configuration for all three backbone models under the matched Run-003 + Run-005×2 recipe. Values are drawn from available training reports and benchmark artifacts.

Table 9.1: Complete LoRA fine-tuning hyperparameters across three backbones.

Parameter	**Qwen3.5-9B**	**Qwen3.6-27B**	**Gemma-4-31B**
`max_seq_length`	4096	4096	4096
`num_train_epochs`	4.0	4.0	4.0
`learning_rate`	2×10^{-4}	2×10^{-4}	2×10^{-4}
`per_device_train_batch_size`	1	1	1
`gradient_accumulation_steps`	4	4	4
`lora_r`	16	16	16
`lora_alpha`	16	16	16
`lora_dropout`	0.0	0.0	0.0
`seed`	3407	3407	3407
`load_in_4bit`	True	True	True
`warmup_ratio`	0.1	0.1	0.1
`weight_decay`	0.01	0.01	0.01
`train_rows`	928	928	928
`eval_rows`	0	0	0
Backbone-specific			
Vision LoRA	enabled	enabled	disabled
LoRA target modules	vision+lang	vision+lang	all-linear
Chat template	Qwen default	Qwen default	gemma-4
GPU memory util.	0.6	0.6	0.95
Training outcomes			
Train runtime (s)	6419	12619	8050
Train loss (final)	0.2156	0.1068	0.0474

Note: training loss values are not directly comparable across backbones due to differ-

ences in tokenizer behaviour, chat-template structure, target-sequence distribution, and model parameterisation.

9.2 8-Fact V1 Baseline Benchmark

The earliest complete benchmark uses an 8-fact LoRA adapter trained on Run-001 (180 tasks) and evaluated on the Run-002 holdout (50 samples). This line predates the 13-fact ontology and is included for historical reference.

Table 9.2: 8-fact v1 benchmark: Qwen3.5-9B Base vs. LoRA-v1 on Run-002 holdout (50 samples, EN).

Metric	**Base**	**LoRA-v1**	**Δ**
JSON valid rate	1.0	1.0	—
Schema valid rate	1.0	1.0	—
Fact accuracy	0.7600	0.9150	+0.1550
Macro F_1	0.4241	0.5847	+0.1605
Seen F_1	0.4714	0.8380	+0.3666
Sample exact match	0.06	0.36	+0.30
Critical false positives	13	15	+2

The LoRA adapter improves fact-level metrics substantially but increases critical false positives from 13 to 15, confirming the 8-fact v1 ontology's structural weakness: compound targets such as `ins_go` were not visually verifiable from single frames and produced systematic false-positive errors on completion-type facts.

9.3 Dataset Fact Distributions

Table 9.3 reports per-fact `seen` counts for the two main training sources (Run-003, Run-005) and the two 13-fact holdout sets (Run-002 newfacts, Run-004 random). Counts are drawn from the corresponding `stats.json` files.

The Run-005 composition rebalance successfully improved coverage of underrepresented facts: `supt_page_visible` from 20 to 54 combined seen, `fcsmc_page_visible` from 55 to 111, and `ins_grnd_alignment_text_visible` from 58 to 130.

Table 9.3: Per-fact `seen` counts across training and holdout datasets (13-fact v2 ontology).

Fact ID	Run-003 (220)	Run-005 (122)	Run-002 new (50)	Run-
tac_page_visible	46	24	23	
supt_page_visible	20	34	5	
fcs_page_visible	30	40	13	
fcs_page_x_marks_visible	16	18	1	
bit_root_page_visible	18	43	9	
fcsmc_page_visible	55	56	9	
fcsmc_intermediate_result_visible	18	15	1	
fcsmc_in_test_visible	18	19	6	
fcsmc_final_go_result_visible	18	22	2	
hsi_page_visible	106	109	49	
hsi_map_layer_visible	79	37	27	
ins_grnd_alignment_text_visible	58	72	42	
ins_ok_text_visible	12	14	4	

9.4 CLI Command Reference

The `simtutor` package provides the following CLI commands:

`run`. Executes a simulation using a mock environment adapter with a pre-recorded scenario file and writes the event log.

`replay`. Validates an existing event log against a procedure pack, checking step ordering and state-machine consistency.

`score`. Scores an event log using the scoring engine, producing omission counts, state-violation counts, and a total error score.

`record-vlm`. Runs VLM fact extraction on a set of vision observations and records predictions for benchmarking.

`replay-bios`. Replays a raw DCS-BIOS capture through the telemetry pipeline and optionally invokes the help generation loop.

`validate`. Validates JSONL files against a named schema from the schema registry.

All commands accept model configuration overrides (`-model-provider`, `-model-name`, `-model-base-url`, `-model-timeout-s`) and a language switch (`-lang`).

9.5 Model Provider Configuration

Model access is configured through environment variables: `SIMTUTOR_MODEL_PROVIDER` (`openai_compat` | `ollama` | `stub`), `SIMTUTOR_MODEL_BASE_URL`, `SIMTUTOR_MODEL_NAME`, `SIMTUTOR_MODEL_TIMEOUT_S`, `SIMTUTOR_MODEL_API_KEY` (required for `openai_compat`), `SIMTUTOR_MODEL_ENABLE_MULTIMODAL` (toggles base64 image encoding in requests), and `SIMTUTOR_LANG` (`zh` | `en`). The live runtime validates all required configuration at startup and redacts API keys from all log output.

9.6 Replay Evaluation Suite

Table 9.4 lists the five replay evaluation cases in the `fa18c_startup_v04` suite.

Table 9.4: Replay evaluation cases.

Case ID	Expected step	Vision	Expected target
`noop_2min`	S01	unavailable	`battery_switch`
`batteryon_2min`	S02	unavailable	`fire_test_switch`
`Batteryon_enginGenoff_2min`	S01	unavailable	`generator_left_switch`
`fcs_reset_fcs_bit_2min`	S18	available	`fcs_bit_switch`
`ins_2min`	S12	available	`ins_mode_knob`

Cases with `vision_status = available` include per-frame sidecar files (`frames.jsonl`) in the corresponding DCS Saved Games directory, enabling replay of vision-dependent procedure steps without a live DCS instance.

9.7 Supplementary Architecture and Export Diagrams

The main chapters use simplified diagrams to support the research argument. Figures 9.1, 9.2, and 9.3 retain additional engineering detail for reproducibility and operational context.

9.8 VR Pilot Study — Supplementary Data

Table 9.5 lists participant experience profiles.

Table 9.6 lists the specific steps not completed by each without-tutor participant.

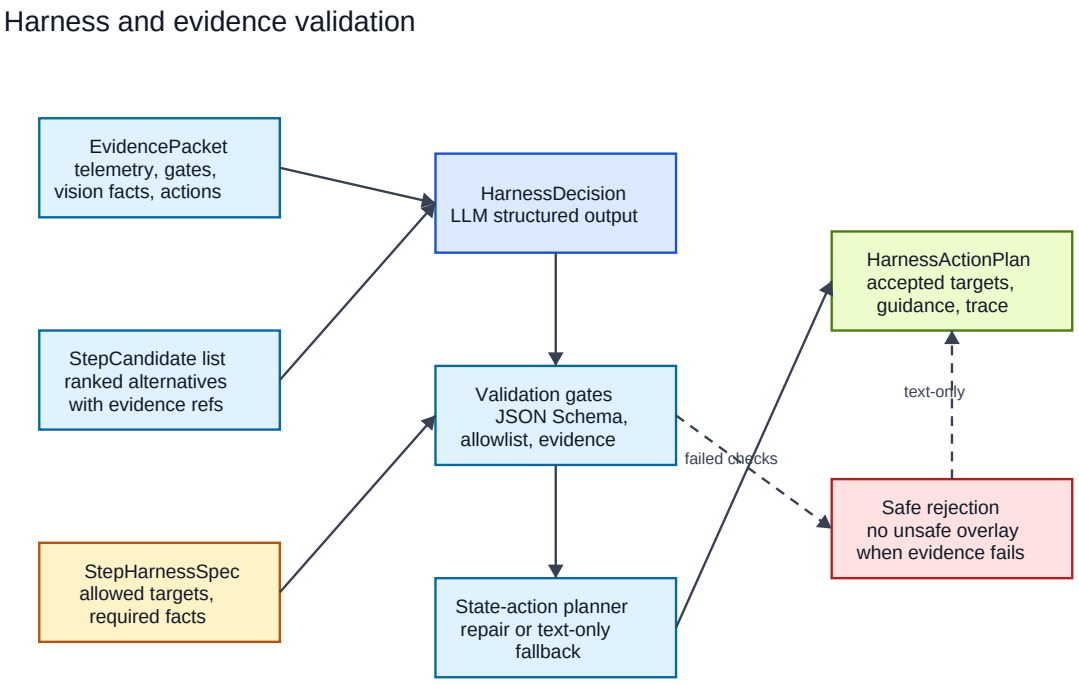

Figure 9.1: Harness and evidence validation. Candidate tutor actions are checked against schema constraints, UI allowlists, evidence consistency, and procedure state before an overlay action plan is accepted; unsafe or under-grounded actions are repaired, rejected, or downgraded to text-only guidance.

Table 9.5: Pilot participant experience profiles (self-reported).

P	Condition	DCS exp.	Flight sim.	VR exp.	Quest 3	Group
P01	without_tutor	frequent	frequent	frequent	frequent	intermediate
P02	without_tutor	intermediate	intermediate	frequent	frequent	—
P03	without_tutor	novice	novice	intermediate	frequent	beginner
P04	with_tutor	intermediate	intermediate	frequent	frequent	intermediate
P05	with_tutor	intermediate	intermediate	frequent	frequent	—
P06	without_tutor	novice	novice	intermediate	frequent	beginner
P07	without_tutor	novice	novice	intermediate	frequent	beginner
P08	with_tutor	novice	novice	frequent	frequent	—
P09	with_tutor	novice	novice	intermediate	intermediate	beginner

— = not recorded.

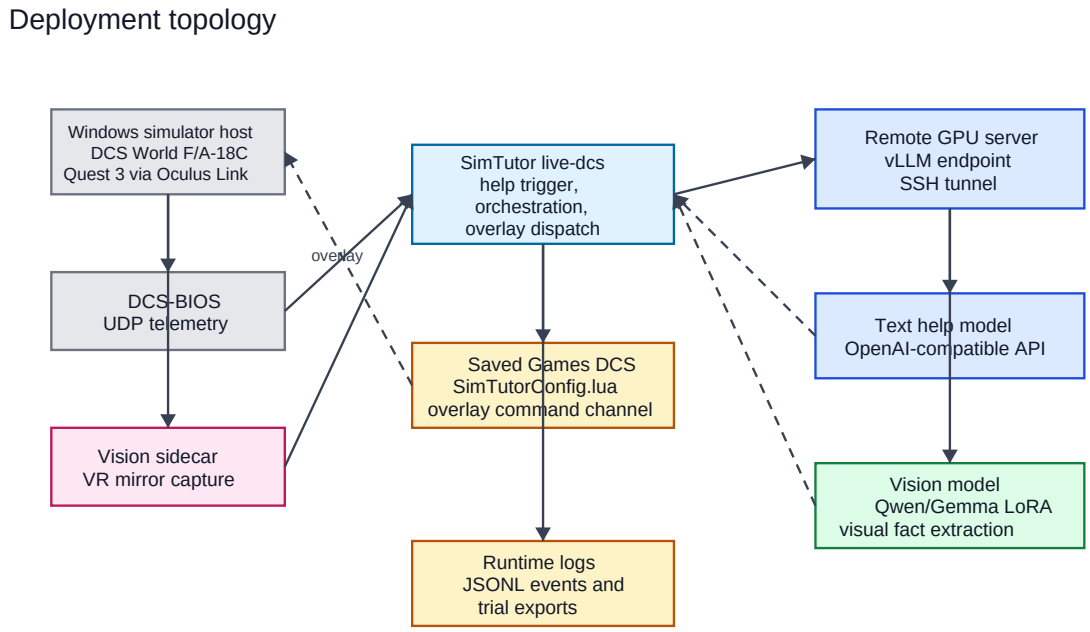

Figure 9.2: Deployment topology used by the prototype. The simulator, Quest 3 overlay path, local runtime, replay utilities, and remote model providers are separated so that live trials and offline analysis can use the same evidence and logging contracts.

Table 9.6: Steps not completed in the without-tutor condition (manual coding).

P	Steps not completed
P01	S09, S12, S18, S19, S20, S22, S24, S27, S29, S31
P02	S02, S14, S18, S19, S27, S30
P03	S07, S09, S16, S18, S19, S26, S27, S29, S32
P06	S07, S18, S19, S27
P07	S02, S09, S12, S13, S16, S17, S18, S19, S22, S26, S27, S28, S29, S32

S18 (FCS-MC BIT page on right DDI) and S19 (FCS BIT final GO result) are missed by all five without-tutor participants. S27 (flap switch to AUTO) is missed by all five. These steps are vision-priority and require either VLM confirmation or instructor knowledge of the correct display-page sequence.

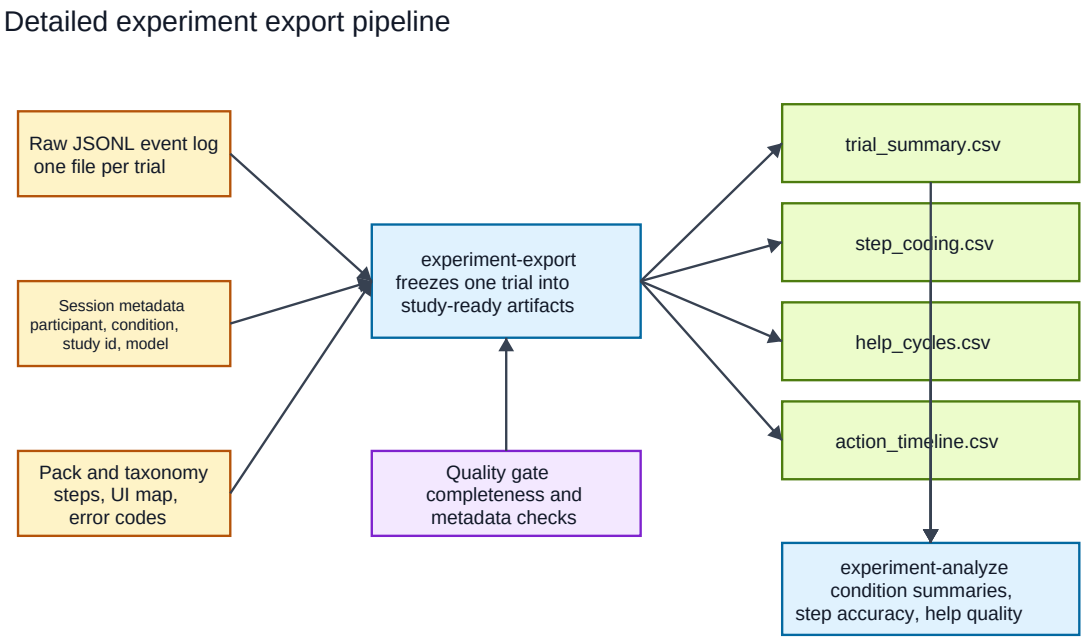

Figure 9.3: Detailed experiment export pipeline. Runtime event logs and trial metadata are transformed into participant-level summaries, step coding, help-cycle records, quality gates, and study-level analysis tables.

Table 9.7 reports the auto-coded error counts (before manual review) alongside the manual error counts, illustrating the magnitude of auto-to-manual correction in this pilot.

Table 9.7: Auto-coded vs. manual error counts per participant.

P	Condition	Auto OM	Auto CO	Manual OM	Manual CO	Total manual
P01	without_tutor	4	3	8	0	10
P02	without_tutor	0	3	0	0	1
P03	without_tutor	0	4	7	0	11
P06	without_tutor	0	3	2	0	4
P07	without_tutor	1	9	11	0	11
P04	with_tutor	0	4	0	3	7
P05	with_tutor	0	3	0	3	6
P08	with_tutor	0	3	0	0	0
P09	with_tutor	0	3	0	3	5

Auto-coded errors are from the automated pipeline (Auto_Error_* columns in `step_coding.csv`). Manual errors are from the human-reviewed columns (Error_*). P08's auto errors were all overridden to zero by manual review (see data-quality note in Section 6.3).